

Developer Portal and API Documentation

Functional Scope and Screen Specifications

TICVAI

Developer and API Management

Table of Contents

Board 1 — Developer Portal & API Documentation	4
Screen 1 — Developer Portal Home	5
Screen 2 — API Catalog	7
Screen 3 — API Endpoint Documentation	8
Screen 4 — Request & Response Explorer	9
Screen 5 — API Schema & OpenAPI Viewer	11
Screen 6 — Interactive API Explorer	12
Screen 7 — SDK Library	13
Screen 8 — Code Samples Library	14
Screen 9 — Postman Collection Center	16
Screen 10 — Developer Help & Documentation Center	17
Board 2 — Developer Accounts & API Credentials	20
Screen 1 — Developer Account Directory	20
Screen 2 — Developer Registration & Onboarding	22
Screen 3 — Developer Profile	23
Screen 4 — Organization Account Management	25
Screen 5 — Developer Teams & Members	26
Screen 6 — Developer Roles & Permissions	27
Screen 7 — API Key Management	29
Screen 8 — Client Secret Management	30
Screen 9 — OAuth & Access Token Management	31
Screen 10 — Credential Revocation & Security History	32
Board 3 — API Gateway, Security & Access Control	37
Screen 1 — API Gateway Command Center	38
Screen 2 — API Access Policy Builder	39
Screen 3 — API Scope & Permission Management	41
Screen 4 — Module API Licensing	42
Screen 5 — Tenant API Access Configuration	44
Screen 6 — Rate Limit Management	45
Screen 7 — Throttling Policy Management	46
Screen 8 — IP Allowlist & Denylist	47
Screen 9 — API Security Monitoring	48
Screen 10 — API Security & Access Audit	50
Board 4 — API Monitoring & Analytics	54
Screen 1 — API Operations Dashboard	55
Screen 2 — API Usage Analytics	56
Screen 3 — API Consumption Monitoring	57
Screen 4 — API Performance Monitoring	58
Screen 5 — Latency Analysis	59
Screen 6 — API Error Monitoring	60
Screen 7 — API Success & Failure Analysis	62
Screen 8 — Integration Health Monitor	63
Screen 9 — Developer Adoption Analytics	64
Screen 10 — API Alerts & Incident Drilldown	66
Board 5 — Webhooks & Event Management	71
Screen 1 — Event & Webhook Command Center	72
Screen 2 — Business Event Catalog	73
Screen 3 — Webhook Registration	75
Screen 4 — Webhook Configuration	77
Screen 5 — Event Subscription Management	78

Screen 6 — Event Routing & Filtering.....	80
Screen 7 — Webhook Delivery Monitor	81
Screen 8 — Retry & Dead-Letter Management.....	83
Screen 9 — Event Replay Center	84
Screen 10 — Event History, Retention & Audit	85
Board 6 — API Lifecycle & Release Management.....	91
Screen 1 — API Lifecycle Command Center	92
Screen 2 — API Version Registry	93
Screen 3 — Version Configuration.....	94
Screen 4 — API Change Management	96
Screen 5 — Breaking Change Detection	97
Screen 6 — Deprecation Management	99
Screen 7 — Migration Management.....	100
Screen 8 — Developer Change Notifications.....	101
Screen 9 — API Release Notes.....	103
Screen 10 — API Release History & Audit	104
Board 7 — Sandbox & Integration Testing	110
Screen 1 — Sandbox Command Center	110
Screen 2 — Sandbox Environment Provisioning.....	112
Screen 3 — Tenant Sandbox Management	114
Screen 4 — Environment Clone & Reset.....	115
Screen 5 — Test Data Management.....	117
Screen 6 — Synthetic Data Generator	119
Screen 7 — API & Webhook Testing Workspace	120
Screen 8 — Performance & Regression Testing.....	122
Screen 9 — Sandbox Access, Isolation & Data Masking.....	123
Screen 10 — Sandbox Monitoring, Usage & Lifecycle.....	125
Board 8 — Business API Coverage & Partner Ecosystem	131
Screen 1 — Business API Coverage Matrix.....	132
Screen 2 — Ticketing & Reservation API Coverage	134
Screen 3 — Membership, Loyalty & Wallet API Coverage	135
Screen 4 — Access & Resource API Coverage	137
Screen 5 — CRM & Customer API Coverage	138
Screen 6 — F&B, Retail & Commerce API Coverage	139
Screen 7 — Pricing, Reporting & AI API Coverage.....	141
Screen 8 — Partner Developer Portal	142
Screen 9 — Integration Certification Program	144
Screen 10 — API & Integration Marketplace.....	146

Board 1 — Developer Portal & API Documentation

Module: 13 — Developer & API Management

Board: 1 of 8 within Developer & API Management

Screens: 10

Board Type: Developer Experience / API Discovery & Documentation

TICVAI Developer Portal
Build amazing experiences with TICVAI APIs
Discover, integrate and innovate with our comprehensive suite of APIs for ticketing, attractions, membership, retail, and more.

Search APIs, endpoints, documentation, examples...

Popular searches: ticketing, reservations, payments, membership, AI, webhooks

Quick Start
Everything you need to get started with TICVAI APIs

- Explore APIs: Browse our API catalog
- Getting Started: Start your integration journey
- API Explorer: Test APIs interactively
- SDKs: Get the latest SDKs
- Code Samples: Ready-to-use examples
- Postman Collections: Download collections
- Integration Guides: Step-by-step tutorials
- Help & Support: Get developer assistance

API Modules
Explore our complete API ecosystem

- Ticketing: Manage ticket products and sales
- Products: Products and inventory management
- Pricing: Dynamic and seasonal pricing
- Promotions: Discounts and campaigns
- Orders: Order management and fulfillment
- Payments: Payment processing and refunds
- Membership: Membership programs
- Loyalty: Loyalty points and rewards
- Wallet: Digital wallet and credits
- Access Control: Ticket validation and access
- Events: Event management
- Reservations: Reservations and bookings
- Resources: Resource scheduling
- Seating: Seating and venue management
- CRM: Customer management
- F&B: Food and beverage
- Retail: Retail operations
- Reporting: Analytics and reporting
- AI Services: AI and recommendation services
- More Modules: Coming Soon

Recent Updates

- New API: AI Recommendation API (Mar 15, 2024)
- Updated: Ticketing API v2.1 (Mar 12, 2024)
- New SDK: Python SDK v3.2 (Mar 10, 2024)
- Deprecated: Legacy Reporting API v1 (Mar 8, 2024)
- Documentation: Webhook Guide Updated (Mar 5, 2024)

Your Activity

- Recently Viewed: Create Ticket Product (2 hours ago)
- Favorites: Search Availability (5 hours ago)
- Recommended: Membership API (1 day ago)
- Payment Processing (2 days ago)
- AI Recommendations (3 days ago)

Developer Resources

- API Status
- Release Notes
- Migration Guides
- Certification

Need Help? Get support from our developer team. Contact Support

TICVAI © 2024 TICVAI. All rights reserved. Privacy | Terms | Support | UAE (GMT+4)

1. Developer Portal Home
Build amazing visitor experiences
Integrate with TICVAI's powerful APIs for ticketing, reservations, membership, retail, and more.

2. API Catalog
Discover and explore all TICVAI APIs

API Name	Module	Version	Type	Status	Access
Ticketing API	Ticketing	v2	REST	Active	Public
Products API	Products	v2	REST	Active	Partner
Payments API	Payments	v1	REST	Active	Partner
Membership API	Membership	v1	REST	Active	Partner
Loyalty API	Loyalty	v1	REST	Active	Partner
Events API	Events	v2	REST	Active	Public
Access Control API	Access Control	v1	REST	Active	Partner
AI Recommendations API	AI Services	v1	REST	Active	Partner

3. API Endpoint Documentation
Create Ticket Product
POST /v2/ticket-products

4. Request & Response Explorer
Request: Create Standard Admission Ticket
Response: 201 Created
Error: 400 Bad Request

5. API Schema & OpenAPI Viewer
Ticket Product Schema
Detailed data model and validation rules

6. Interactive API Explorer
Try the API
Parameters: POST /v2/ticket-products
Body: { "product": "Standard Admission", "quantity": 1 }
Authentication: Basic (username, password)

7. SDK Library
TICVAI SDKs
Official SDKs to help you integrate faster

8. Code Samples Library
Code Samples
Quickstart code snippets for various languages and frameworks

9. Postman Collection Center
Postman Collections
Download and import TICVAI APIs with Postman

10. Developer Help & Documentation Center
Developer Documentation
Search, reference, and support resources

Board Purpose

Provide a centralized **TICVAI Developer Portal** where internal developers, clients, partners, resellers, system integrators, and approved third parties can discover, understand, explore, test, and implement TICVAI APIs.

The portal shall act as the primary technical entry point into the TICVAI integration ecosystem and shall provide structured API documentation, endpoint specifications, request/response examples, schemas, interactive API exploration, SDKs, code samples, Postman collections, and implementation guidance.

The board shall be **version-aware, tenant-aware, module-aware and permission-aware**.

It must not independently manage API credentials, gateway policies, API monitoring, webhooks, API lifecycle configuration, or sandbox provisioning. Those functions are controlled by subsequent Developer & API Management boards.

Screen 1 — Developer Portal Home

Purpose

Provide developers with a single landing page for discovering TICVAI APIs, documentation, SDKs, tools, recent updates, and implementation resources.

UI / Configuration

Header

- TICVAI Developer Portal
- Global API search
- Environment indicator
- Current documentation version
- Developer/account profile
- Notifications

Quick Start Cards

- Explore APIs
- Getting Started
- API Explorer
- SDKs
- Code Samples
- Postman Collections
- Integration Guides
- Help & Support

API Module Directory

Display available API families such as:

- • Ticketing
- • Products
- • Pricing
- • Promotions

- • Orders
- • Payments
- • Membership
- • Loyalty
- • Wallet
- • Access Control
- • Events
- • Reservations
- • Resources
- • Seating
- • CRM
- • F&B
- • Retail
- • Reporting
- • AI Services

Developer Overview

Show:

- • Recently viewed APIs
- • Favorite APIs
- • Recently updated APIs
- • Recommended documentation
- • Quick-start guides

Recent Updates

- New API
- Updated endpoint
- New version
- Deprecated endpoint
- New SDK
- Documentation update

Actions

[Explore APIs](#) | [Open Quick Start](#) | [Search Documentation](#) | [View SDKs](#) | [Open API Explorer](#)

Rules / Acceptance Criteria

- Search shall work across APIs, endpoints, schemas and documentation.
- Only APIs visible to the user's role/organization shall appear.
- Documentation shall clearly identify version and status.
- No API secrets shall be displayed on this screen.

Screen 2 — API Catalog

Purpose

Provide a searchable and filterable catalog of all TICVAI API capabilities.

UI / Configuration

Filters

- Business Module
- API Category
- REST / Event / Webhook
- API Version
- Status
- Public / Partner / Internal
- Tenant availability
- Licensed / Not Licensed

API Cards / Table

Each API should show:

- **API Name**
Ticketing API
- **Module**
Ticketing Catalogue
- **Version**
v2
- **Status**
Active
- **Type**
REST
- **Access**
Partner

Description

Create, retrieve and manage ticketing-related objects.

Available Operations

GET | POST | PUT | PATCH | DELETE

API Status

Support:

- • Draft
- • Beta
- • Active
- • Deprecated
- • Retired

Actions

[View API](#) | [View Endpoints](#) | [Open Explorer](#) | [View Schema](#) | [View Examples](#) | [Favorite](#)

Rules

The Catalog is a **discovery layer**, not API-access configuration.

Whether the tenant actually has permission to execute an API is controlled by later API security/licensing boards.

Screen 3 — API Endpoint Documentation

Purpose

Provide complete technical documentation for an individual TICVAI API endpoint.

Example

Create Ticket Product

POST /v1/ticket-products

UI / Configuration

Endpoint Header

- Method
- Endpoint path
- API family
- API version
- Status
- Description

Authentication Information

- Authentication type
- Required OAuth scope
- Required permissions

Do not display actual credentials.

Request Configuration

- Path parameters
- Query parameters
- Headers
- Request body
- Required/optional indicator
- Data type
- Validation requirements

Response Documentation

- HTTP status
- Response schema
- Response example

Support documented responses such as:

- • 200 Success
- • 201 Created
- • 400 Validation Error
- • 401 Unauthorized
- • 403 Forbidden
- • 404 Not Found

- • 409 Conflict
- • 429 Rate Limited
- • 500 Server Error
- **Technical Behaviors**
 - Where applicable:
 - • • Pagination
 - • • Filtering
 - • • Sorting
 - • • Idempotency
 - • • Localization
 - • • Currency handling
 - • • Date/time handling
 - • • Rate-limit information

Actions

[Try in Explorer](#) | [Copy Endpoint](#) | [View Schema](#) | [View Examples](#) | [View Related APIs](#)

Acceptance Criteria

Every published endpoint must contain sufficient documentation for an external developer to implement it without relying on undocumented TICVAI behavior.

Screen 4 — Request & Response Explorer

Purpose

Help developers understand how TICVAI APIs behave through structured request and response examples.

UI / Configuration

Use Case Selector

For example:

- Ticket Product API:
 - • • Create Standard Admission
 - • • Create Open-Dated Ticket
 - • • Create Timeslot Ticket
 - • • Create Reserved Seat Ticket
 - • • Create Membership Product
- • Selecting a use case loads the relevant example.

Request Panel

Show:

- • Headers
- • Parameters
- • JSON payload
- • Authentication placeholder

Response Panel

Show:

- Success response
- Validation response
- Error response
- Response headers

Code Language Selector

Where supported:

- cURL
- JavaScript
- Python
- Java
- C#
- PHP
- Node.js

Features

- Syntax highlighting
- Copy payload
- Copy response
- Copy code
- Expand/collapse objects
- Download example
- Switch API version

Rule

Example credentials/tokens must always use safe placeholders and must never expose real credentials.

Screen 5 — API Schema & OpenAPI Viewer

Purpose

Provide structured visibility into TICVAI API schemas and machine-readable API definitions.

UI / Configuration

Schema Navigation

API

→ Endpoint

→ Request Object

→ Response Object

→ Nested Objects

Schema Viewer

Display:

Field	Type	Required	Description
productId	UUID	Yes	TICVAI product identifier

Field	Type	Required	Description
name	String	Yes	Product name
status	Enum	Yes	Product lifecycle status
currency	String	Conditional	Transaction currency

Schema Features

Support:

- Objects
- Arrays
- Enums
- Required fields
- Nullable fields
- Format
- Minimum/maximum
- Pattern
- Relationships
- Example values
- Validation constraints

OpenAPI

Support:

- OpenAPI/Swagger specification
- View raw specification
- Download specification
- Version selection
- Schema comparison

Version Comparison

Example:

v1 → v2

Highlight:

- Added field
- Removed field
- Changed type
- New required field
- Deprecated field

Actions

[Download OpenAPI](#) | [Copy Schema](#) | [Compare Version](#) | [Open Endpoint](#)

Screen 6 — Interactive API Explorer

Purpose

Allow developers to safely test TICVAI API requests directly from the Developer Portal.

UI / Configuration

Environment

Default:

Sandbox

Production execution must not be the default behavior.

Endpoint Selector

Example:

POST /v1/ticket-products

Request Builder

Allow:

- Path parameters
- Query parameters
- Headers
- Request body
- Authentication context

Execute

Send Request

Result

Show:

- HTTP status
- Response body
- Response headers
- Request duration
- Timestamp
- Correlation/request ID
- Example:

201 Created

Response Time: 184 ms

Validation

Before execution:

- ✓ Required parameters complete
- ✓ Payload schema valid
- ✓ Authentication available
- ✓ Environment available

Safety

Destructive or production actions must be restricted according to TICVAI security policy.

Sandbox provisioning itself is **not configured here**; that belongs to Board 7.

Screen 7 — SDK Library

Purpose

Provide officially supported TICVAI development SDKs.

UI / Configuration

SDK Catalog

Examples:

- JavaScript / TypeScript
- Python
- Java
- .NET
- PHP
- Mobile SDKs where applicable
- Each SDK displays:
 - SDK name
 - Current version
 - Supported API versions
 - Release date
 - Status
 - Compatibility
 - Documentation

SDK Detail

Provide:

- Installation
- Authentication setup
- Initialization
- Basic request example
- Error handling
- Pagination
- Webhook handling where relevant
- Upgrade instructions

Version History

Example:

TICVAI Node SDK

v3.2 — Current

v3.1 — Supported

v2.x — Deprecated

Actions

[View SDK](#) | [Install Instructions](#) | [View Examples](#) | [View Release Notes](#) | [Download / Package Link](#)

Screen 8 — Code Samples Library

Purpose

Provide reusable integration examples based on common TICVAI business scenarios.

UI / Configuration

Search / Filters

- Module
- API
- Language
- Use case
- Difficulty
- API version

Example Use Cases

- Create Ticket
- Search Availability
- Create Reservation
- Process Payment
- Validate Ticket
- Create Membership
- Add Wallet Credit
- Retrieve Customer
- Generate Report
- Retrieve AI Recommendation

Sample Detail

Show:

- **Use Case**
Create a Timeslot Reservation
- **Language**
Node.js
- **API Version**
v2

Required APIs

- Availability
- Reservation
- Ticketing

Then provide:

- • Prerequisites
- • Dependencies

- • Example code
- • Expected response
- • Error handling
- • Related documentation

Actions

[Copy](#) | [Download](#) | [Open API](#) | [Open in Explorer](#)

Screen 9 — Postman Collection Center

Purpose

Allow developers to quickly test TICVAI APIs using maintained Postman collections and environment templates.

UI / Configuration

Collections

Organize by:

- • Complete TICVAI API
- • Ticketing
- • Commerce
- • Membership & Loyalty
- • Access Control
- • Resources
- • CRM
- • Reporting
- • AI

Collection Information

Each collection shows:

- • Collection version
- • API version
- • Last updated
- • Included endpoints
- • Compatibility status

Environment Templates

Support templates such as:

Sandbox Environment

- Base URL
- Tenant ID
- Client ID placeholder
- Access token placeholder
- Venue ID
- API version

No real credentials should be packaged.

Version History

Allow developers to identify which Postman collection corresponds with which API version.

Actions

[Download Collection](#) | [Import/Open Where Supported](#) | [Download Environment](#) | [View Documentation](#) | [View Change History](#)

Screen 10 — Developer Help & Documentation Center

Purpose

Provide the central knowledge and support area for developers implementing TICVAI integrations.

UI / Configuration

Getting Started

- Platform overview
- Authentication overview
- First API call
- Sandbox guide
- Error handling
- API conventions

Integration Guides

Examples:

- • Ticketing Integration
- • POS Integration
- • Access Control Integration
- • Membership Integration
- • Wallet Integration
- • CRM Integration
- • Payment Integration
- • Reporting Integration
- • AI Integration

Technical Guides

- Authentication
- Pagination
- Idempotency
- Localization
- Currency
- Time zones
- Error handling
- Retry strategy

- Webhooks
- Version migration

Troubleshooting

Searchable issues such as:

- 401 Unauthorized
- 403 Scope Missing
- 409 Configuration Conflict
- 429 Rate Limit Exceeded

Help Navigation

Provide links to:

- • Developer Support
- • API Status
- • Release Notes
- • Migration Guides
- • Certification
- • Partner Resources

Detailed support/certification workflows belong to later boards.

Cross-Board Boundaries

This is important for Soft Labs because **Board 1 must not become the whole Developer Management module.**

Function	Owner
API documentation & discovery	Board 1
Developer registration/accounts	Board 2
API keys/client secrets/OAuth management	Board 2
API scopes/security policies	Board 3
Rate limits/throttling/IP policies	Board 3
API monitoring/performance/errors	Board 4
Webhook configuration/delivery	Board 5
Version/deprecation/release management	Board 6
Sandbox provisioning/data/reset	Board 7
Partner/certification/API marketplace	Board 8

Board 1 may **display or link to** information from these areas but shall not duplicate their management interfaces.

Board-Level Business Rules & Acceptance Criteria

- 1. Documentation synchronization:** Published API documentation should be generated or synchronized from controlled API definitions wherever technically feasible to prevent documentation from differing from the implemented API.
 - 2. Version awareness:** Every endpoint, schema, SDK, example and Postman collection must identify the API version to which it applies.
 - 3. Module awareness:** APIs must be grouped according to TICVAI business modules rather than presented as one unstructured endpoint list.
 - 4. Permission awareness:** Developers shall only discover restricted documentation when their organization/account has the appropriate access.
 - 5. Tenant awareness:** Where API behavior differs by tenant configuration or licensing, the portal should clearly indicate availability without turning Board 1 into the licensing configuration interface.
 - 6. Search:** Global search must cover API names, endpoint paths, descriptions, schemas, documentation, examples and guides.
 - 7. Safe testing:** Interactive API execution shall default to an approved sandbox environment. Production execution shall be governed separately.
 - 8. Security:** Credentials, tokens, secrets and production customer information must never be embedded in documentation, examples, SDK samples or downloadable collections.
 - 9. Traceability:** Documentation publishing and changes should be traceable to version/release information.
 - 10. Consistency:** SDKs, code examples, OpenAPI definitions and Postman collections must correspond to supported API versions.
-

Board 1 — Final Screen Map

Developer Portal & API Documentation

- Screen 1:** Developer Portal Home
- Screen 2:** API Catalog
- Screen 3:** API Endpoint Documentation
- Screen 4:** Request & Response Explorer
- Screen 5:** API Schema & OpenAPI Viewer
- Screen 6:** Interactive API Explorer
- Screen 7:** SDK Library
- Screen 8:** Code Samples Library
- Screen 9:** Postman Collection Center
- Screen 10:** Developer Help & Documentation Center

This gives Soft Labs a clean **10-screen Board 1** while preserving the boundaries for Boards 2–8.

Board 2 — Developer Accounts & API Credentials

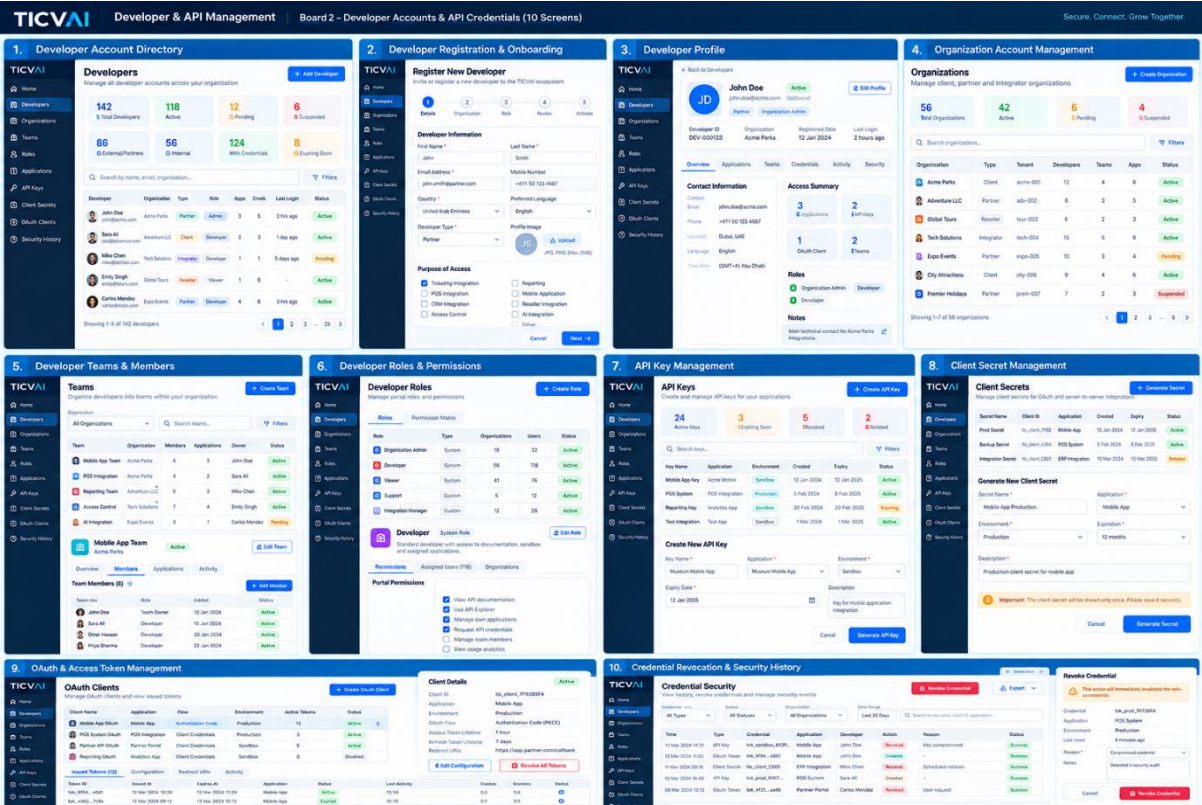
Module: 13 — Developer & API Management

Board: 2 of 8

Screens: 10

Board Type: Developer Identity / Organization / Credential Administration

Board Purpose



Provide TICVAI with a centralized administration layer for managing **developers, partner organizations, development teams, roles, API applications, API keys, client secrets, OAuth credentials, access tokens, and credential revocation.**

Board 2 establishes **who the developer is** and **what technical credentials have been issued to them.**

It does **not** define detailed API authorization policies, module licensing, rate limits, throttling, IP restrictions, or API gateway security. Those belong to **Board 3 — API Gateway, Security & Access Control.**

The intended hierarchy is:

Organization → Developers → Teams → Applications → Credentials → Authorized API Access

Screen 1 — Developer Account Directory

Purpose

Provide administrators with a central directory of all developer accounts that have access to the TICVAI developer ecosystem.

UI / Configuration

Header KPIs

- Total Developers
- Active Developers

- Pending Registration
- Suspended Developers
- External/Partner Developers
- Internal Developers
- Developers with Active Credentials
- Developers with Expiring Credentials

Search & Filters

- Developer Name
- Email
- Organization
- Developer Type
- Status
- Role
- Registration Date
- Last Login
- Credential Status
- Tenant

Developer Table

Columns:

Developer	Organization	Type	Role	Applications	Credentials	Last Login	Status
-----------	--------------	------	------	--------------	-------------	------------	--------

Developer Types

Support:

- Internal
- Client
- Partner
- Reseller
- System Integrator
- Third Party

Status

- Pending
- Active
- Suspended
- Locked
- Disabled

Actions

[Add Developer](#) | [View Profile](#) | [Edit](#) | [Suspend](#) | [Reactivate](#) | [View Applications](#) | [View Credentials](#)

Rules / Acceptance Criteria

- Developer identity must be unique.

- Status changes must be audited.
 - Suspending a developer must follow configurable security rules regarding their access.
 - Credentials must not be exposed directly from the directory.
-

Screen 2 — Developer Registration & Onboarding

Purpose

Manage the complete onboarding process for a new developer.

UI / Configuration

Registration Source

- Self-registration
- Admin invitation
- Partner invitation
- Organization invitation
- Internal provisioning

Developer Information

- First Name
- Last Name
- Email
- Mobile
- Country
- Preferred Language
- Developer Type

Organization

- Existing Organization
- Create New Organization
- Independent Developer where permitted

Requested Purpose

Examples:

- • Ticketing Integration
- • POS Integration
- • CRM Integration
- • Access Control
- • Reporting
- • Mobile Application
- • Reseller Integration
- • AI Integration
- • Other

Onboarding Progress

Example:

Registration → Verification → Organization → Role → Approval → Activated

Approval Information

Show:

- Submitted By
- Submitted Date
- Reviewer
- Review Status
- Approval Notes

Actions

[Invite Developer](#) | [Approve](#) | [Reject](#) | [Request Information](#) | [Save Draft](#) | [Activate](#)

Rules

Registration does not automatically grant API access.

A developer can be successfully registered while having:

API Access: None

until the appropriate access is granted.

Screen 3 — Developer Profile

Purpose

Provide a complete administrative profile for an individual developer.

UI / Configuration

Profile Header

Show:

- Developer Name
- Developer ID
- Organization
- Developer Type
- Status
- Primary Role
- Registration Date
- Last Login

Tabs

[Overview](#) | [Applications](#) | [Teams](#) | [Credentials](#) | [Activity](#) | [Security](#)

Overview

Show:

- Contact details
- Organization
- Assigned teams

- Assigned roles
- Active applications
- Active credentials
- Preferred environment
- Developer notes

Access Summary

Example:

Applications: 3

API Keys: 2

OAuth Clients: 1

Teams: 2

Security Summary

- Last Login
- Failed Login Attempts
- Last Credential Created
- Last Credential Revoked
- Account Status

Actions

[Edit Profile](#) | [Assign Team](#) | [Assign Role](#) | [View Credentials](#) | [Suspend Account](#)

Rule

The profile can summarize permissions but detailed API security policy configuration remains in Board 3.

Screen 4 — Organization Account Management

Purpose

Manage organizations participating in the TICVAI developer ecosystem.

Examples include:

- TICVAI client
- Venue operator
- Technology partner
- Payment provider
- Reseller
- System integrator
- Government integration partner

UI / Configuration

Organization Directory

Columns:

- Organization
- Organization ID
- Type

- • Tenant
- • Developers
- • Teams
- • Applications
- • Status

Organization Profile

General

- Organization Name
- Legal Name
- Organization ID
- Organization Type
- Country
- Primary Contact

TICVAI Relationship

- Tenant
- Client
- Partner
- Reseller
- Integration Provider

Developer Statistics

- Developers
- Active Developers
- Applications
- Active Credentials

Organization Status

- Pending
- Active
- Suspended
- Disabled

Actions

[Create Organization](#) | [Edit](#) | [Add Developer](#) | [Add Team](#) | [View Applications](#) | [Suspend](#)

Rules

Organization-level isolation must be enforced.

Developers from Organization A must not see Organization B's applications, credentials, or developer information unless explicitly authorized.

Screen 5 — Developer Teams & Members

Purpose

Allow organizations to organize developers into teams for easier administration and controlled access.

Example

Organization: Dubai Attraction Group

Teams:

Mobile App Team

POS Integration Team

Reporting Team

Access Control Integration Team

UI / Configuration

Team Directory

Team	Organization	Members	Applications	Team Role	Status
------	--------------	---------	--------------	-----------	--------

Team Detail

General

- Team Name
- Description
- Organization
- Team Owner
- Status

Members

Show:

- • Developer
- • Role
- • Added Date
- • Status

Application Associations

Example:

Mobile App Team

Applications:

- • Dubai Attraction Mobile App
- • Customer Wallet Integration

Actions

Create Team | Add Member | Remove Member | Assign Owner | Associate Application | Disable Team

Rules

- Team membership must not automatically bypass API security.
- Removing a developer from a team must trigger access-impact validation.

- Team membership changes must be audited.
-

Screen 6 — Developer Roles & Permissions

Purpose

Define what developers are allowed to do within the **Developer Portal itself**.

This distinction is critical.

This screen manages:

Developer Portal permissions

It does not replace:

API runtime scopes and gateway authorization — Board 3.

Example Roles

Organization Administrator

Can:

- • Manage organization developers
- • Create teams
- • Manage applications
- • Request credentials

Developer

Can:

- • View documentation
- • Use approved sandbox
- • Manage assigned applications
- • Request/rotate own credentials where permitted

Viewer

Can:

- • View documentation
- • View applications

UI / Configuration

Role Directory

- Role
- Type
- Organizations
- Users
- Status

Permission Matrix

Permission categories:

- • Developer Management

- Team Management
- Application Management
- Credential Management
- Sandbox Access
- Documentation Access
- Usage Visibility

Actions

Create Role | Clone Role | Edit Permissions | Assign Developer | Disable Role

Acceptance Criteria

Portal permissions and runtime API permissions must remain independently governed.

Screen 7 — API Key Management

Purpose

Create and manage API keys associated with approved developer applications.

UI / Configuration

Header KPIs

- Active Keys
- Expiring Soon
- Revoked
- Rotated
- Applications Without Keys

API Key Directory

Key Name	Application	Organization	Environment	Created	Expiry	Last Used	Status
----------	-------------	--------------	-------------	---------	--------	-----------	--------

Create API Key

Fields:

- Key Name
- Application
- Organization
- Environment
- Expiry Date
- Description

Generated Credential

When generated:

API Key Created Successfully

The full credential shall be displayed only according to TICVAI's secure credential policy, preferably **one time only**.

Example:

tvk_sandbox_.....8X2P

Status

- Active
- Expiring
- Expired
- Revoked
- Rotated

Actions

[Create Key](#) | [Rotate](#) | [Revoke](#) | [View Usage Reference](#) | [Copy Once](#)

Rules

- Never store/display secrets as ordinary readable text.
 - Production and sandbox credentials must be distinguishable.
 - Key creation/revocation must be audited.
 - Revoked keys cannot be restored.
 - Rotation should support controlled migration where policy permits.
-

Screen 8 — Client Secret Management**Purpose**

Securely manage client credentials used by server-to-server TICVAI integrations.

UI / Configuration**Client/Application Header**

Example:

Application: Museum ERP Integration

Client ID: tic_client_7F92...

Environment: Production

Secret Directory

Show:

- • Secret Name
- • Client ID
- • Created Date
- • Expiry
- • Last Rotated
- • Status
- Never show the existing secret value.

Generate Secret

Configuration:

- • Secret Name
- • Expiration
- • Rotation Policy
- • Application

- Environment

Secret Creation

After generation:

Save this client secret now. It will not be shown again.

Rotation

Support:

Current Secret → New Secret → Grace Period → Old Secret Revoked

where permitted by security policy.

Actions

Generate Secret | Rotate Secret | Revoke | View History

Rules

- Secret values must be encrypted/securely stored.
- Existing secrets must not be retrievable as plaintext.
- Every creation, rotation and revocation must be audited.

Screen 9 — OAuth & Access Token Management

Purpose

Manage OAuth client configuration and provide visibility/control over tokens issued to approved applications.

UI / Configuration

OAuth Client Directory

Show:

- Client
- Application
- Organization
- OAuth Flow
- Environment
- Token Policy
- Status

Supported OAuth Configuration

Where applicable:

- Client Credentials
- Authorization Code
- PKCE
- Refresh Tokens

Actual supported flows shall depend on TICVAI security architecture.

OAuth Client Detail

Client Information

- Client ID
- Application
- Organization
- Environment

Token Configuration

- Access Token Lifetime
- Refresh Token Lifetime
- Refresh Allowed
- Token Rotation
- Revocation Behavior

Redirect URIs

Where applicable.

Token Visibility

Show metadata such as:

- • Token Identifier
- • Application
- • Issued At
- • Expires At
- • Status
- • Last Activity
- Do **not** display raw access tokens.

Actions

Create OAuth Client | Edit Configuration | Revoke Token | Revoke All Application Tokens

Boundary

API scopes may be displayed for reference, but detailed scope/access-policy configuration belongs to **Board 3**.

Screen 10 — Credential Revocation & Security History

Purpose

Provide a centralized security workspace for reviewing credential history and immediately revoking compromised, obsolete, or unauthorized developer credentials.

UI / Configuration

Security KPIs

- Active Credentials
- Expiring Soon
- Recently Rotated

- Recently Revoked
- Suspicious Credential Activity

Credential Search

Search across:

- • API Keys
- • Client Secrets
- • OAuth Clients
- • Tokens
- Filters:
 - • • Organization
 - • • Developer
 - • • Application
 - • • Environment
 - • • Credential Type
 - • • Status
 - • • Creation Date
 - • • Expiration
 - • • Last Used

Credential History

Example timeline:

10:42 — API key created
11:03 — First API usage
14:18 — Rotation requested
14:21 — Replacement key created
15:00 — Original key revoked

Revocation Workflow

Admin selects:

Revoke Credential

System displays impact:

Application: Museum Mobile App

Environment: Production

Credential: API Key

Last Used: 8 minutes ago

Connected integration may stop functioning immediately.

Require:

- • Revocation Reason
- • Confirmation
- • Optional Incident/Case Reference

Bulk Emergency Revocation

Where authorized:

- **Revoke all credentials for:**
 - • Developer

- • • Application
- • • Organization
- • This must require elevated permissions and confirmation.

Actions

Revoke | Rotate | View Activity | View Application | Export Security History

Acceptance Criteria

- Revocation must take effect according to the platform's security architecture.
- Security-sensitive actions must be immutable in audit history.
- Raw secret values must never appear in history.
- Emergency bulk revocation requires appropriate permissions.
- Revocation impact must be clearly shown before execution.

Important Object Relationship

Soft Labs should design Board 2 around this hierarchy:

Organization

↓

Developer Accounts

↓

Teams & Roles

↓

Applications

↓

API Keys / Client Secrets / OAuth Clients

↓

Tokens

There is one additional concept that should appear throughout Board 2 even though it does not need a separate screen:

Developer Application

Credentials should **not normally be issued directly against a human developer**.

For example:

Developer: John Smith

Organization: Dubai Museum

Application: Dubai Museum Mobile App

Environment: Production

OAuth Client: Mobile-App-Production

This gives TICVAI much cleaner governance and traceability.

Board 2 — Cross-Board Boundaries

Function	Owner
Developer identity	Board 2
Organization accounts	Board 2

Function	Owner
Teams	Board 2
Developer Portal roles	Board 2
Applications	Board 2
API keys	Board 2
Client secrets	Board 2
OAuth clients/tokens	Board 2
Credential rotation/revocation	Board 2
API scopes & runtime permissions	Board 3
Module/API licensing	Board 3
Rate limits & throttling	Board 3
IP allowlist/denylist	Board 3
Security monitoring	Board 3
API usage/performance analytics	Board 4
Sandbox provisioning	Board 7

Board 2 — Final Screen Map

Board 2 — Developer Accounts & API Credentials

1. Developer Account Directory
2. Developer Registration & Onboarding
3. Developer Profile
4. Organization Account Management
5. Developer Teams & Members
6. Developer Roles & Permissions
7. API Key Management
8. Client Secret Management
9. OAuth & Access Token Management
10. Credential Revocation & Security History

This gives Board 2 a clean progression from **identity** → **organization** → **team** → **permission** → **credential** → **security lifecycle**, without taking functionality from Board 3.

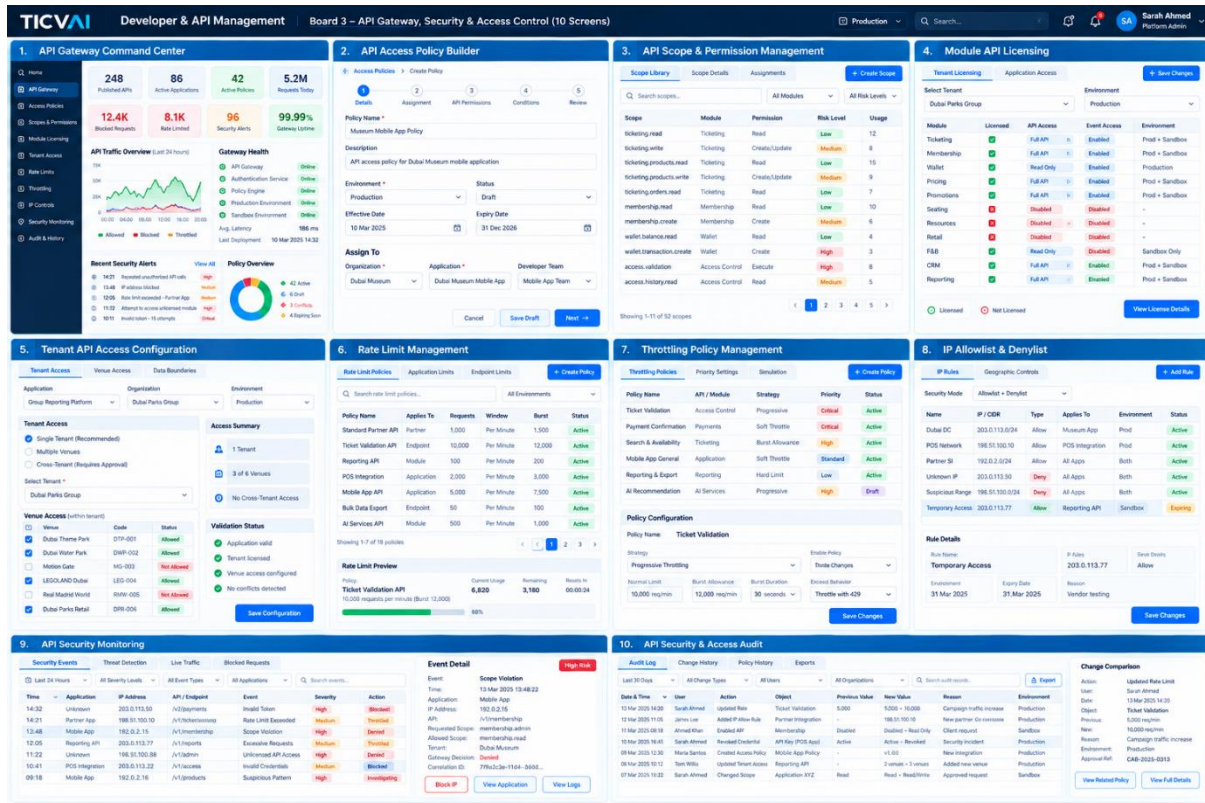
Board 3 — API Gateway, Security & Access Control

Module: 13 — Developer & API Management

Board: 3 of 8

Screens: 10

Board Type: API Security / Authorization / Gateway Governance



Board Purpose

Provide TICVAI with the centralized control layer for governing **which APIs, modules, operations, tenants, applications, and integrations** may be accessed and under what security conditions.

Board 2 established:

Who is connecting → Organization → Developer → Application → Credential

Board 3 now determines:

What can they access → Which APIs → Which operations → Which tenant/venue → Under what limits → From where → With what security controls

The intended architecture is:

Application / Credential

↓

API Access Policy

↓

Scopes & Permissions

↓

Module / Tenant Entitlement

↓

Rate Limit / Throttling

↓

Network & IP Controls

↓

API Gateway

↓

TICVAI Business APIs

This board must not duplicate credential issuance from Board 2, API operational analytics from Board 4, or general commercial subscription management.

Screen 1 — API Gateway Command Center**Purpose**

Provide administrators and security teams with one centralized overview of TICVAI API gateway configuration, access status, security policies, and immediate security issues.

UI / Configuration**Header KPIs**

- Published APIs
- Active API Applications
- Active Access Policies
- Active API Credentials
- Requests Today
- Blocked Requests
- Rate-Limited Requests
- Security Alerts

Gateway Health

Display:

- • Gateway Status
- • Production Status
- • Sandbox Status
- • Average Latency
- • Authentication Service
- • Policy Engine
- • Last Configuration Deployment

API Traffic Overview

Visualize:

- • Allowed Requests
- • Denied Requests
- • Throttled Requests
- • Authentication Failures

Policy Health

Show:

- • Active Policies
- • Draft Policies
- • Policies with Conflicts

- • Expiring Temporary Access
- • Unassigned Applications

Security Alerts

Examples:

- Repeated unauthorized API calls
- Application exceeded rate threshold
- Requests received from blocked IP
- Application attempted access to an unlicensed API module

Actions

[View Policies](#) | [Review Alerts](#) | [Manage API Access](#) | [Review Gateway Health](#)

Acceptance Criteria

The screen is primarily an operational/configuration overview. Detailed historical API performance analytics remain in Board 4.

Screen 2 — API Access Policy Builder

Purpose

Create reusable policies defining the API access allowed to an application, partner, integration, or organization.

Policy Structure

Example:

- **Policy:** Museum Mobile Application
- **Applies To:**
Dubai Museum Mobile App
- **Tenant:**
Dubai Museum
- **Environment:**
Production
- **Allowed APIs:**
 - • • Ticketing
 - • • Availability
 - • • Reservations
 - • • Membership
- • **Denied APIs:**
 - • • • Finance
 - • • • Staff Management
 - • • • System Administration

UI / Configuration

Policy Information

- Policy Name
- Description
- Environment

- Status
- Effective Date
- Expiry Date where applicable

Policy Assignment

- Organization
- Application
- Developer Team where applicable
- Integration
- Tenant

API Permissions

Allow selection by:

- • Module
- • API
- • Endpoint group
- • Operation

Policy Effects

Support:

Allow

Deny

Explicit deny should take precedence according to TICVAI security policy.

Actions

Create Policy | Clone | Validate | Test | Save Draft | Activate | Suspend

Validation

Before activation:

- ✓ Application exists
- ✓ Credential available
- ✓ APIs licensed
- ✓ Scopes valid
- ✓ No policy conflicts
- ✓ Tenant boundaries valid

Screen 3 — API Scope & Permission Management

Purpose

Define fine-grained runtime permissions for TICVAI APIs.

This is different from the **Developer Portal permissions** configured in Board 2.

Board 2:

- Can the developer manage applications or credentials?
- Board 3:
- Can the application execute `ticketing.products.write`?

UI / Configuration

Scope Library

Example scopes:

Ticketing

- ticketing.read
- ticketing.write
- ticketing.products.read
- ticketing.products.write
- ticketing.orders.read

Membership

- membership.read
- membership.create
- membership.update

Wallet

- wallet.balance.read
- wallet.transaction.create

Access Control

- access.validation
- access.history.read

Scope Detail

Show:

- • Scope Name
- • Business Module
- • Description
- • Associated APIs
- • Associated Operations
- • Risk Classification
- • Applications Using Scope

Permission Levels

Where applicable:

- • Read
- • Create
- • Update
- • Delete
- • Execute
- • Approve
- • Admin

Risk Classification

Example:

- **Low**
Read availability
- **Medium**
Create reservation
- **High**
Issue refund
- **Critical**
Financial/admin operations

Actions

Create Scope | Edit | Assign | Remove | View Impact

Rules

High-risk scopes may require additional approval before assignment.

Screen 4 — Module API Licensing

Purpose

Control which TICVAI business-module APIs are available to a tenant, client, partner, or application based on licensed capabilities.

This implements the important matrix requirement that TICVAI can expose selected business APIs while excluding others.

Example

A customer licenses:

- ✓ Ticketing
 - ✓ Membership
 - ✓ Wallet
- but does not license:
- ✗ Resource Management
- ✗ Seating
- ✗ Retail

Their integration should therefore not gain those API capabilities simply because the endpoints technically exist.

UI / Configuration

Tenant / Organization Selector

API Module Matrix

Module	Licensed	API Enabled	Event Access	Environment
Ticketing	✓	✓	✓	Prod + Sandbox
Membership	✓	✓	✓	Prod + Sandbox
Wallet	✓	✓	✓	Production
Seating	✗	✗	✗	—

Module	Licensed	API Enabled	Event Access	Environment
Resources	×	×	×	—

Configuration

Support:

- API Enabled
- API Disabled
- Read Only
- Full API
- Event Access
- Sandbox Only
- Production Allowed

Actions

[Enable API](#) | [Disable API](#) | [Change Access](#) | [Review License](#) | [View Impact](#)

Important Boundary

Board 3 enforces the API entitlement.

The actual commercial subscription/license package remains owned by TICVAI's **Subscription & Licensing Module**.

Screen 5 — Tenant API Access Configuration

Purpose

Enforce multi-tenant and venue-level boundaries for API access.

TICVAI is multi-tenant, so an API credential must never gain access to another tenant's data merely because it has a valid API scope.

UI / Configuration

Access Context

- Organization
- Application
- Tenant
- Venue
- Environment

Tenant Access

Example:

- **Application:** Group Reporting Platform
- Allowed:
 - ✓ Tenant — Dubai Parks Group
 - Within tenant:

- • ✓ Venue A
- ✓ Venue B
- ✕ Venue C

Data Access Boundary

Configure where applicable:

- • Tenant
- • Venue
- • Business Unit
- • Legal Entity
- • Region
- • Sales Channel

Cross-Tenant Access

Default:

Disabled

Exceptional cross-tenant integrations must require explicit authorization and elevated approval.

Actions

Assign Tenant | Add Venue | Remove Venue | Request Cross-Tenant Access | Validate Access

Rules

Tenant isolation must be enforced at runtime, not only through the user interface.

Screen 6 — Rate Limit Management

Purpose

Configure maximum API request rates to protect TICVAI services and ensure fair consumption.

UI / Configuration

Rate Limit Policies

Columns:

- • Policy
- • API/Application
- • Environment
- • Requests
- • Window
- • Burst
- • Status

Example

Standard Partner API

1,000 requests / minute

Ticket Validation API

10,000 requests / minute

Reporting API

100 requests / minute

Configuration Dimensions

Rate limits may apply by:

- • Organization
- • Application
- • API
- • Endpoint
- • Credential
- • Tenant
- • Environment

Limit Windows

Examples:

- • Per second
- • Per minute
- • Per hour
- • Per day

Response Behavior

When limit is exceeded:

HTTP 429 — Too Many Requests

Where supported, expose:

- • Limit
- • Remaining
- • Reset Time
- • Retry-After

Actions

[Create Limit](#) | [Edit](#) | [Clone](#) | [Assign](#) | [Disable](#) | [View Usage](#)

Screen 7 — Throttling Policy Management

Purpose

Control API traffic behavior when consumption approaches or exceeds permitted limits or when TICVAI must protect platform stability.

Rate limiting answers:

How much traffic is permitted?

Throttling answers:

How should TICVAI control traffic when demand becomes excessive?

UI / Configuration

Throttling Strategies

Support where technically applicable:

- • Hard Limit
- • Soft Throttle
- • Burst Allowance
- • Progressive Throttling
- • Temporary Restriction
- • Priority Traffic

Example

Ticket Validation API

Normal:

10,000 requests/minute

Burst:

12,000/minute for 30 seconds

Beyond burst:

Throttle/reject according to policy.

Priority Classification

Examples:

Critical

- Ticket Validation
- Payment Confirmation
- Access Control

Standard

- Reservation
- CRM

Background

- Reporting
- Bulk Export

Actions

Create Policy | Set Priority | Configure Burst | Simulate | Activate

Rule

Critical real-time operational APIs may use different throttling behavior from reporting or bulk-data APIs.

Screen 8 — IP Allowlist & Denylist

Purpose

Control API access based on trusted or prohibited network addresses.

UI / Configuration

Network Security Mode

Options:

- No IP Restriction
- Allowlist Only
- Denylist
- Allowlist + Denylist

IP Rules

Support:

- IPv4
- IPv6
- CIDR range where technically supported

Rule Information

- Rule Name
- IP / CIDR
- Organization
- Application
- Environment
- Action
- Effective Date
- Expiry
- Reason
- Status

Example

POS ERP Integration

Production:

- Allowed:
203.0.113.0/24
- Everything else:
Denied

Temporary Rules

Support temporary IP authorization with automatic expiration.

Actions

[Add IP](#) | [Add Range](#) | [Block IP](#) | [Temporarily Allow](#) | [Remove](#) | [View History](#)

Rules

- Changes must be audited.
- Production IP changes may require approval.
- Invalid/overlapping rules should be detected.
- Deny rules should take precedence according to security policy.

Screen 9 — API Security Monitoring

Purpose

Provide security-focused monitoring of API access and identify suspicious or unauthorized API behavior.

This screen focuses specifically on **security events**, not general API performance analytics.

Security Indicators

- Authentication Failures
- Authorization Failures
- Invalid Tokens
- Revoked Credential Attempts
- Blocked IP Attempts
- Scope Violations
- Tenant Boundary Violations
- Excessive Requests
- Suspicious API Patterns

Security Event Table

Time	Application	API	Event	Risk	Action
10:32	Partner App	Payments	Invalid Token	Medium	Blocked
10:34	Mobile App	Finance	Scope Violation	High	Denied
10:41	Unknown	Login API	Excessive Attempts	Critical	Blocked

Risk Levels

- Informational
- Low
- Medium
- High
- Critical

Event Drilldown

Show:

- • Timestamp
- • Organization
- • Application
- • Credential reference
- • API
- • Endpoint
- • IP
- • Tenant
- • Security rule triggered

- Gateway decision
- Correlation ID

Actions

[Investigate](#) | [Block IP](#) | [Revoke Credential](#) | [Suspend Application](#) | [View Audit](#)

Credential revocation can deep-link to Board 2.

Boundary

General fraud/customer risk analysis belongs to the TICVAI Fraud module, not this board.

Screen 10 — API Security & Access Audit

Purpose

Maintain a complete, searchable history of API access-policy and security configuration changes.

UI / Configuration

Audit KPIs

- Policy Changes
- Permission Changes
- API Licensing Changes
- IP Rule Changes
- Rate Limit Changes
- Security Actions

Audit Timeline

Track:

- Policy Created
- Scope Assigned
- API Module Enabled
- Tenant Access Changed
- Rate Limit Modified
- IP Address Blocked
- Application Suspended

Audit Record

Each record should include:

- Date/Time
- User
- Organization
- Action
- Object
- Previous Value
- New Value
- Reason
- Approval Reference

- Source
- Correlation ID where applicable

Before / After Comparison

Example:

Rate Limit

Before:

1,000 requests/min

After:

5,000 requests/min

Changed By:

Admin User

Reason:

Expected campaign traffic

Filters

- Date
- User
- Organization
- Application
- API
- Change Type
- Risk
- Environment

Actions

[View Detail](#) | [Compare Changes](#) | [Export Audit](#) | [View Related Policy](#)

Rules / Acceptance Criteria

- Security audit history must be immutable to ordinary administrators.
- Sensitive secret values must never be recorded in readable audit logs.
- Every production security-policy change must be traceable.
- Export capability must respect administrator permissions.

Board 3 — Core Security Evaluation Flow

For Soft Labs, every API request should conceptually pass through controls similar to:

1. Is the credential valid?

↓

2. Is the application active?

↓

3. Is this API/module licensed?

↓

4. Does the application have the required scope?

↓

5. Is the tenant/venue allowed?

↓

6. Is the source IP permitted?

↓

7. Is the request within rate/throttle limits?

↓

8. Are security policies satisfied?

↓

ALLOW API REQUEST

If any mandatory control fails:

DENY / THROTTLE + LOG SECURITY EVENT

This runtime principle is one of the most important requirements of Board 3.

Cross-Board Boundaries

Function	Owner
Developer accounts	Board 2
API keys / secrets / OAuth credentials	Board 2
Credential rotation/revocation	Board 2
API access policies	Board 3
Runtime API scopes	Board 3
Module API entitlement	Board 3
Tenant/venue API boundaries	Board 3
Rate limits	Board 3
Throttling	Board 3
IP restrictions	Board 3
API security events	Board 3
API usage/performance analytics	Board 4
Webhook management	Board 5
API lifecycle/versioning	Board 6
Sandbox provisioning	Board 7
Commercial subscription/license definition	Existing Subscription & Licensing Module
Customer/payment/ticket fraud	Existing/new Fraud Intelligence Boards

Board 3 — Final Screen Map

API Gateway, Security & Access Control

1. API Gateway Command Center
2. API Access Policy Builder
3. API Scope & Permission Management
4. Module API Licensing
5. Tenant API Access Configuration
6. Rate Limit Management
7. Throttling Policy Management
8. IP Allowlist & Denylist
9. API Security Monitoring
10. API Security & Access Audit

This keeps Board 3 focused on the **runtime protection and authorization layer** between TICVAI's developer applications and the actual business APIs.

Board 4 — API Monitoring & Analytics

Module: 13 — Developer & API Management

Board: 4 of 8

Screens: 10

Board Type: API Operations / Consumption / Performance / Integration Intelligence



Board Purpose

Provide TICVAI administrators, technical teams, clients, and authorized partners with centralized visibility into **API traffic, consumption, performance, latency, errors, availability, integration health, developer adoption, and operational incidents.**

Board 4 answers questions such as:

Which APIs are being used the most?

Which client or application is consuming them?

Are APIs performing within expected SLA?

Which endpoints are slow?

Which integrations are failing?

Is the problem on TICVAI's side or the partner side?

Which developers and partners are actively adopting our APIs?

The board shall support drill-down from:

Platform → Tenant → Organization → Application → API → Endpoint → Individual Request

It must remain separate from **Board 3 security monitoring**. Board 3 identifies unauthorized or suspicious API access; Board 4 focuses on **operational health, usage and performance**.

Screen 1 — API Operations Dashboard

Purpose

Provide one executive and operational overview of the health and usage of the TICVAI API ecosystem.

Header KPIs

- Total API Requests Today
- Successful Requests
- Failed Requests
- Success Rate
- Average Response Time
- P95 Latency
- Active Applications
- APIs with Incidents

Filters

- Date / Time Range
- Tenant
- Venue
- Organization
- Application
- API Module
- API
- Environment
- API Version

API Traffic Overview

Chart:

Requests over time

Break down by:

- • Successful
- • Failed
- • Throttled
- • Other

API Health Overview

Show APIs such as:

API	Requests	Success	Avg. Latency	Health
Ticketing	1.8M	99.98%	145 ms	Healthy
Payments	620K	99.91%	212 ms	Healthy
Access	2.4M	99.99%	72 ms	Healthy
Reporting	88K	96.2%	1.8 sec	Degraded

Operational Alerts

Examples:

Reporting API latency above threshold

Partner integration failure rate increased to 8.4%

Reservation API success rate below SLA

Actions

[Open API](#) | [Investigate Failure](#) | [View Performance](#) | [View Integration](#) | [View Incident](#)

Acceptance Criteria

The dashboard must support drill-down without forcing administrators to navigate separately through every API.

Screen 2 — API Usage Analytics

Purpose

Analyze API usage across the entire TICVAI ecosystem.

Header KPIs

- Requests
- Unique Applications
- Active Organizations
- Active Tenants
- Active APIs
- Requests per Application

Usage Dimensions

Analyze by:

- API
- Business Module
- Tenant
- Venue
- Organization
- Application
- Developer
- Environment
- API Version
- Time Period

API Usage Ranking

Example:

API	Requests	% of Total	Applications	Trend
Access Validation	2.4M	31%	42	↑
Ticketing	1.8M	23%	65	↑
Availability	1.1M	14%	38	→

API	Requests	% of Total	Applications	Trend
Payments	620K	8%	21	↓

Usage Trends

Support:

- Hourly
- Daily
- Weekly
- Monthly

Heatmap

Show usage by:

Day of Week × Hour

This helps identify API peak periods.

Actions

[Drill Down](#) | [Compare](#) | [Export](#) | [View Application](#) | [View API](#)

Screen 3 — API Consumption Monitoring

Purpose

Monitor actual API consumption against configured entitlements, quotas, allowances, or commercial consumption plans.

This is different from Board 3 rate limiting.

Rate Limit:

How quickly can requests be made?

Consumption:

How much API usage has occurred during the billing/usage period?

Header KPIs

- Current Period Requests
- Included Allowance
- Consumed
- Remaining
- Overages
- Forecasted Period Usage

Consumption Table

Organization	Application	API	Used	Allowance	Usage %
Dubai Museum	Mobile App	Ticketing	820K	1M	82%
Partner X	Reseller	Availability	1.2M	1M	120%
Venue Y	ERP	Reporting	60K	100K	60%

Thresholds

Support:

- 50%
- 75%
- 90%
- 100%
- Custom

Forecast

Example:

- Current Consumption: **82%**
- Expected Month-End: **108%**
- Estimated Overage: **80,000 requests**

Actions

[View Consumption](#) | [Export](#) | [Configure Alert](#) | [View Subscription](#)

Commercial allowance definitions remain in Subscription & Licensing.

Screen 4 — API Performance Monitoring

Purpose

Measure operational performance and availability of TICVAI APIs.

Header KPIs

- Average Response Time
- P50
- P95
- P99
- Availability
- Throughput
- Slow Requests
- Degraded APIs

Performance Table

API	Avg	P95	P99	Availability	Throughput	Status
Access	72 ms	110 ms	180 ms	99.999%	2.8K/sec	Healthy
Ticketing	145 ms	280 ms	420 ms	99.99%	1.2K/sec	Healthy
Reporting	1.8s	3.2s	6.4s	99.4%	24/sec	Degraded

Performance Trend

Graph:

- Response Time
- Request Volume

- Error Rate
- Allow overlay to identify relationships.

SLA / SLO Status

Example:

Ticket Validation

Target: <200 ms P95

Current: 110 ms

Within Target

Actions

[View Endpoint](#) | [Compare Period](#) | [Investigate](#) | [View Latency](#)

Screen 5 — Latency Analysis

Purpose

Provide detailed analysis of where API response time is being consumed and identify slow endpoints.

Latency Ranking

Rank:

Slowest APIs

Slowest Endpoints

Most Degraded Applications

Endpoint Table

Endpoint	Avg	P50	P95	P99	Requests	Trend
/reports/export	2.4s	1.8s	5.1s	8.3s	12K	↑
/availability	320ms	210ms	620ms	1.1s	890K	↑

Latency Breakdown

Where telemetry supports it:

- **Gateway**
 - Authentication
 - TICVAI Service
 - Database
 - External Dependency
 - Response
 - Example:
 - Gateway: 18 ms
- Authentication: 12 ms
- Ticketing Service: 84 ms
- Database: 190 ms
- Response: 8 ms

Total: 312 ms

Comparison

Compare:

- Current vs Previous Period
- API Version
- Tenant
- Application
- Environment

Actions

Investigate Trace | View Endpoint | Compare | Create Alert

Screen 6 — API Error Monitoring

Purpose

Centralize API errors and help technical teams identify recurring failures.

Header KPIs

- Errors Today
- Error Rate
- 4xx Errors
- 5xx Errors
- New Error Types
- Affected Applications

Error Categories

Support:

- 400 — Validation
- 401 — Authentication
- 403 — Authorization
- 404 — Not Found
- 409 — Conflict
- 429 — Rate Limit
- 500 — Internal Error
- 502/503 — Service Availability

Error Table

Error	API	Endpoint	Count	Applications	Trend	Severity
400 Validation	Ticketing	Create Product	1,820	14	↑	Medium
500 Internal	Reporting	Export	242	8	↑	High
409 Conflict	Reservation	Create	168	11	→	Medium

Error Detail

Show:

- • Error Code
- • Error Message
- • API
- • Endpoint
- • API Version
- • Application
- • Tenant
- • Timestamp
- • Correlation ID
- • Request metadata
- • Response metadata
- Sensitive payload data must be masked.

Actions

[Investigate](#) | [View Requests](#) | [View Application](#) | [Create Alert](#) | [Export](#)

Screen 7 — API Success & Failure Analysis

Purpose

Analyze API success and failure patterns from a business and technical perspective rather than simply listing errors.

Success Funnel

Example:

1,000,000 Requests

↓ 98.8%

988,000 Successful

↓ 1.2%

12,000 Failed

Failure distribution:

- • Validation — 5,200
- • Authentication — 1,800
- • Authorization — 900
- • Business Rule — 2,400
- • Platform Error — 1,200
- • External Dependency — 500

Analysis Dimensions

Compare by:

- • API
- • Endpoint
- • Application
- • Partner
- • Tenant

- • Version
- • Time
- • Environment

Failure Trend

Identify:

New Failure Pattern

Reservation failures increased 340% following Partner App v4.2 deployment.

Failure Ownership

Where determinable:

Client-side

Example:

Invalid payload

TICVAI-side

Example:

Internal service error

External dependency

Example:

Payment provider unavailable

Actions

[Investigate Pattern](#) | [View Application](#) | [View API](#) | [Compare Version](#)

Screen 8 — Integration Health Monitor

Purpose

Provide a consolidated health view of each external or internal integration using TICVAI APIs.

This is especially important because clients will have multiple integrations operating simultaneously.

Integration Health Cards

Examples:

- **Dubai Museum Mobile App**
Healthy — 99.98%
- **Reseller Partner A**
Degraded — 94.2%
- **ERP Integration**
Healthy — 99.9%
- **Payment Provider X**
Incident — 82.1%

Integration Health Score

Calculate from factors such as:

- • API success rate
- • Error rate
- • Latency

- • Availability
- • Credential validity
- • Recent activity
- Security status may be referenced from Board 3.

Integration Detail

Show:

Application

Organization

Environment

APIs Used

Request Volume

Success Rate

Latency

Last Successful Request

Last Failure

Current Incident

Dependency Map

Example:

Museum Mobile App

- Availability API ✓
- Ticketing API ✓
- Payments API ⚠
- Wallet API ✓
- Membership API ✓

Actions

[Open Integration](#) | [Investigate](#) | [View Errors](#) | [View Performance](#) | [Contact Owner](#)

Screen 9 — Developer Adoption Analytics

Purpose

Measure adoption and engagement with TICVAI's developer ecosystem.

Header KPIs

- Registered Developers
- Active Developers
- Active Organizations
- Active Applications
- New Integrations
- Production Integrations

- Sandbox-to-Production Conversion

Adoption Funnel

Registered Developer

↓

Created Application

↓

Created Credential

↓

First Sandbox API Call

↓

Successful Integration

↓

Production API Call

Analytics

Measure:

- New Developers
- Active Developers
- Applications Created
- API Explorer Usage
- SDK Adoption
- Postman Collection Usage
- First Successful API Call
- Time to First API Call
- Sandbox Activity
- Production Adoption

API Adoption

Show:

API	Developers	Applications	Growth	Production Adoption
-----	------------	--------------	--------	---------------------

Partner Adoption

Show which partners are:

- Not Started
- Exploring
- Developing
- Testing
- Production
- Inactive

Actions

[View Developer](#) | [View Organization](#) | [View Application](#) | [Export Analytics](#)

Screen 10 — API Alerts & Incident Drilldown

Purpose

Provide centralized operational alerts and detailed investigation of API incidents.

This is the operational response layer of Board 4.

Alert Types

Support:

- High Error Rate
- High Latency
- Low Availability
- Consumption Threshold
- Traffic Spike
- Traffic Drop
- Integration Failure
- API Degradation
- SLA/SLO Breach
- Security alerts remain in Board 3.

Alert Queue

Alert	API	Impact	Started	Severity	Status
High Latency	Reporting	8 Apps	10:22	High	Investigating
Error Spike	Reservations	Partner A	10:48	Critical	Open
Consumption 90%	Ticketing	Museum	—	Medium	Warning

Incident Detail

Incident

Reservation API Failure Spike

Started

10:48

Affected

- 3 tenants
- 8 applications
- 14 endpoints

Current Error Rate

18.2%

Normal

0.8%

Timeline

10:48 — Threshold exceeded

10:49 — Alert generated

10:51 — Engineering acknowledged

10:55 — Root cause investigation started

11:08 — Payment dependency identified

Related Data

Show:

- • Error trend
- • Latency trend
- • Traffic trend
- • Affected APIs
- • Affected tenants
- • Affected applications
- • Correlation IDs

Actions

[Acknowledge](#) | [Assign](#) | [Investigate](#) | [Add Note](#) | [Resolve](#) | [Export Incident](#)

Acceptance Criteria

- Alerts must be generated from configurable operational thresholds.
- Alerts must support severity and status.
- Incidents must retain investigation history.
- Affected tenants/applications must be identifiable.
- Operational incidents and security incidents must remain distinguishable.

Critical Drill-Down Architecture

Soft Labs should maintain the same context as the administrator moves between screens:

TICVAI Platform

↓

Tenant / Organization

↓

Application / Integration

↓

API

↓

Endpoint

↓

Request / Error / Trace

For example, from Screen 1:

- Reporting API — Degraded
- Clicking it should carry the user into:

Performance → Reporting API

Then:

Endpoint /reports/export

Then:

P95 5.1 sec

Then:

Trace / Correlation ID

The administrator should not have to repeatedly reselect the same tenant, API, application, and time range.

Board 4 — Alert Threshold Configuration

Although we do not need a separate 11th screen, threshold configuration should be available contextually from the relevant monitoring screens.

Examples:

Performance

Alert when:

P95 > 500ms for 5 minutes

Error

Alert when:

5xx > 2% for 3 minutes

Availability

Alert when:

Availability < 99.9%

Consumption

Alert when:

- Usage > 80%
- then:
 - 90%
 - then:
 - 100%

Integration

Alert when:

- No successful request for X minutes
 - where activity is normally expected.
 - This keeps configuration close to the monitored metric.
-

Board 4 — Cross-Board Boundaries

Function	Owner
API credentials	Board 2
API authorization/scopes	Board 3

Function	Owner
Rate-limit configuration	Board 3
Security events	Board 3
API usage analytics	Board 4
Consumption monitoring	Board 4
Performance/latency	Board 4
Error monitoring	Board 4
Success/failure analysis	Board 4
Integration health	Board 4
Developer adoption	Board 4
Operational API alerts/incidents	Board 4
Webhook delivery monitoring	Board 5
API versions/releases	Board 6
Sandbox administration	Board 7
Commercial quota/package definition	Subscription & Licensing
General platform BI	Reporting/Analytics

Board 4 — Final Screen Map

API Monitoring & Analytics

1. API Operations Dashboard
2. API Usage Analytics
3. API Consumption Monitoring
4. API Performance Monitoring
5. Latency Analysis
6. API Error Monitoring
7. API Success & Failure Analysis
8. Integration Health Monitor
9. Developer Adoption Analytics
10. API Alerts & Incident Drilldown

This board gives TICVAI a complete **operational observability layer for the API ecosystem**, while Board 3 remains responsible for protecting that ecosystem.

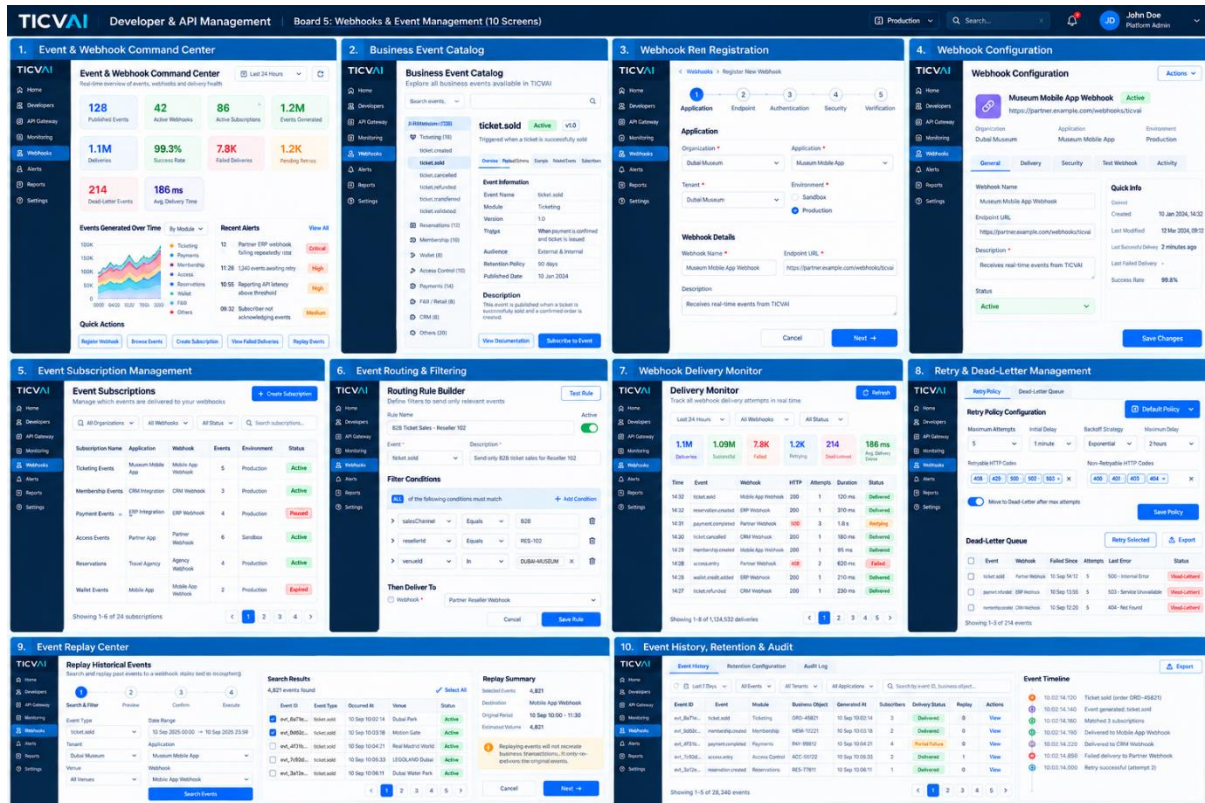
Board 5 — Webhooks & Event Management

Module: 13 — Developer & API Management

Board: 5 of 8

Screens: 10

Board Type: Event-Driven Integration / Webhook Configuration / Delivery & Replay



Board Purpose

Provide TICVAI with a centralized platform for managing **business events**, **webhook endpoints**, **event subscriptions**, **routing/filtering**, **webhook delivery**, **retries**, **failed deliveries**, **event replay**, **retention**, and **event audit history**.

Board 5 enables external systems to receive TICVAI changes in near real time without continuously polling APIs.

For example:

A ticket is sold in TICVAI

→ ticket.sold event is generated

→ Partner subscribes to the event

→ TICVAI sends the event to the configured webhook endpoint

→ Delivery is acknowledged and recorded.

The architecture should be:

TICVAI Business Module

↓

Business Event Generated

↓

Event Catalog

↓

Subscription

↓

Routing / Filtering

↓

Webhook Endpoint

↓

Delivery Attempt

↓

Success / Retry / Dead-Letter

↓

History & Audit

The matrix explicitly requires webhook registration and management, an event catalog, retry and delivery monitoring, plus business-event subscriptions, historical replay, retention, complete audit, filtering/routing, and subscriber health.

Screen 1 — Event & Webhook Command Center

Purpose

Provide one central operational and configuration view of TICVAI's event-driven integration ecosystem.

Header KPIs

- Published Event Types
- Active Webhooks
- Active Subscriptions
- Events Generated Today
- Deliveries Today
- Successful Deliveries
- Failed Deliveries
- Pending Retries
- Dead-Letter Events
- Delivery Success Rate

Filters

- Tenant
- Organization
- Application
- Environment
- Business Module
- Event
- Webhook
- Status
- Time Range

Event Activity

Chart:

Events Generated Over Time

Break down by module:

- Ticketing
- Payments
- Membership
- Loyalty
- Wallet
- Access
- Reservations
- CRM
- F&B/Retail
- Other

Webhook Health

Example:

Webhook	Subscriber	Events	Success	Avg. Delivery	Health
Mobile App	Dubai Museum	12.4K	99.98%	180ms	Healthy
CRM Sync	Venue A	8.2K	99.6%	420ms	Healthy
Partner ERP	Partner X	4.1K	82.3%	2.1s	Critical

Alerts

Examples:

Partner ERP webhook failing repeatedly

1,240 events awaiting retry

Dead-letter queue exceeds threshold

Subscriber has not acknowledged events for 18 minutes

Actions

[Register Webhook](#) | [Browse Events](#) | [Create Subscription](#) | [View Failed Deliveries](#) | [Replay Events](#)

Screen 2 — Business Event Catalog

Purpose

Provide a governed catalog of all business events that TICVAI can publish for integration purposes.

Event Hierarchy

Events should be organized by:

Business Module → Event Domain → Event

For example:

Ticketing

- ticket.created

- ticket.issued
- ticket.sold
- ticket.cancelled
- ticket.refunded
- ticket.transferred
- ticket.validated

Reservation

- reservation.created
- reservation.confirmed
- reservation.modified
- reservation.cancelled
- reservation.expired

Membership

- membership.created
- membership.activated
- membership.renewed
- membership.expired

Wallet

- wallet.credit.added
- wallet.debit.completed
- wallet.transaction.completed

Access

- access.entry
- access.exit
- access.validation.failed

UI / Configuration

Search & Filters

- Module
- Event Category
- Version
- Status
- Audience
- Environment Availability

Event Table

Event	Module	Version	Subscribers	Status
-------	--------	---------	-------------	--------

Event Detail

Show:

- • Event Name
- • Description
- • Trigger
- • Business Module
- • Version
- • Payload Schema
- • Example Payload
- • Required Fields
- • Optional Fields
- • Publication Conditions
- • Retention Policy
- • Related Events

Event Status

- Draft
- Beta
- Active
- Deprecated
- Retired

Actions

[View Event](#) | [View Schema](#) | [View Example](#) | [View Subscribers](#) | [Open Documentation](#)

Boundary

Event version lifecycle must integrate with Board 6 rather than creating an independent version-management framework here.

Screen 3 — Webhook Registration

Purpose

Register the external endpoint to which TICVAI will send subscribed events.

Registration Wizard

Step 1 — Application

Select:

- • Organization
- • Application
- • Tenant
- • Environment

Step 2 — Endpoint

Configure:

- • Webhook Name
- • Endpoint URL
- • Description

- Example:
- <https://partner.example.com/webhooks/ticvai>

Step 3 — Authentication

Support according to approved TICVAI security architecture, for example:

- • Signing Secret
- • Bearer Authentication
- • OAuth where applicable
- • Custom approved authentication

Step 4 — Security

Where supported:

- • Signature validation
- • Secret rotation
- • TLS requirement
- • Allowed destination/security controls

Step 5 — Verification

TICVAI sends a verification/test request.

Status:

Verified ✓

or

Verification Failed

Webhook Status

- Draft
- Pending Verification
- Active
- Suspended
- Disabled
- Failed Verification

Actions

Register | Test Endpoint | Verify | Save Draft | Activate

Rules

- HTTPS should be required for production endpoints.
- Secrets must be securely stored.
- Existing secrets must not be displayed in plaintext.
- Endpoint ownership/verification should occur before activation.
- Registration must be audited.

Screen 4 — Webhook Configuration

Purpose

Manage the technical delivery behavior of an individual webhook after registration.

General Configuration

- Webhook Name
- Endpoint URL
- Application
- Tenant
- Environment
- Status

Delivery Configuration

Configure:

- • HTTP Method where applicable
- • Content Type
- • Connection Timeout
- • Response Timeout
- • Payload Format
- • Event Envelope Version
- • Compression where supported

Security

Show:

- • Authentication Method
- • Signing Enabled
- • Secret Status
- • Last Rotation
- • TLS Status

Delivery Controls

Configure:

- • Maximum Concurrent Deliveries
- • Delivery Ordering where supported
- • Retry Policy Reference
- • Failure Threshold
- • Auto-Suspend Rule
- Example:
- Suspend webhook if 50 consecutive deliveries fail.

Test Webhook

Allow administrator/developer to select:

Test Event

Example:

- ticket.sold

- Then:

Send Test

Show:

- • Request
- • Response
- • HTTP Status
- • Response Time

Actions

[Edit](#) | [Test](#) | [Rotate Secret](#) | [Suspend](#) | [Reactivate](#) | [Disable](#)

Screen 5 — Event Subscription Management

Purpose

Define exactly which business events a webhook/application should receive.

Subscription Structure

Subscriber

Dubai Museum CRM

Webhook

CRM Production Webhook

Subscribed Events

- ✓ ticket.sold
- ✓ ticket.refunded
- ✓ membership.created
- ✓ membership.renewed

UI / Configuration

Subscription Directory

Subscription	Application	Webhook	Events	Environment	Status
--------------	-------------	---------	--------	-------------	--------

Create Subscription

Select by:

- • Module
- • Event Domain
- • Individual Event
- • Event Version

Subscription Scope

Where appropriate:

- • Tenant
- • Venue
- • Business Unit
- • Channel

- Example:
- • Partner may subscribe to:
- • • ticket.sold
- • • but only for:

Venue: Dubai Museum

Status

- Draft
- Active
- Paused
- Expired
- Disabled

Effective Dates

Support:

- • Start Date
- • End Date
- • No Expiry

Actions

Create | Edit | Pause | Resume | Clone | Delete

Rules

An application must not subscribe to events from modules or tenants it is not authorized to access.

Board 3 API/security entitlements should be respected.

Screen 6 — Event Routing & Filtering

Purpose

Allow subscribers to receive only relevant events rather than every event of a selected type.

Example

A reseller subscribes to:

- ticket.sold
- but only wants events where:

salesChannel = B2B

and:

resellerId = RES-102

TICVAI should filter the event before sending it.

UI / Configuration

Routing Rule Builder

IF

Event = ticket.sold

AND

Venue = DUBAI-MUSEUM

AND

Sales Channel = B2B

AND

Reseller ID = RES-102

THEN

Deliver to:

Partner Reseller Webhook

Filter Fields

Depending on event schema:

- • Tenant
- • Venue
- • Product
- • Event
- • Sales Channel
- • Customer Segment
- • Transaction Type
- • Reseller
- • Location
- • Status
- • Amount/Value
- • Custom supported event attribute

Operators

- Equals
- Not Equals
- Contains
- In
- Not In
- Greater Than
- Less Than
- Exists

Routing Priorities

Support priority/order when multiple rules apply.

Testing

Provide:

Sample Event → Run Filter

Result:

✓ Rule Matched

Event will be delivered to 2 subscribers.

Actions

[Create Rule](#) | [Test](#) | [Clone](#) | [Activate](#) | [Disable](#)

Acceptance Criteria

Invalid filters referencing unavailable event fields must be rejected before activation.

Screen 7 — Webhook Delivery Monitor

Purpose

Provide real-time and historical visibility into every webhook delivery attempt.

Header KPIs

- Deliveries
- Successful
- Failed
- Pending
- Retrying
- Average Delivery Time
- P95 Delivery Time

Delivery Table

Time	Event	Webhook	Subscriber	HTTP	Attempts	Duration	Status
------	-------	---------	------------	------	----------	----------	--------

Status

- Pending
- Delivering
- Delivered
- Failed
- Retrying
- Dead-Lettered

Delivery Detail

Show:

- **Event ID**
evt_82A71...
- **Event**
ticket.sold
- **Webhook**
Partner ERP
- **Attempt**
3 of 5
- **HTTP Response**
500

- **Response Time**

1.8 sec

- **Next Retry**

15:42

Request / Response

Allow authorized technical users to inspect:

- • Headers
- • Payload
- • Response
- • Timestamp
- • Correlation ID
- Sensitive information must be masked.

Actions

[View Event](#) | [Retry Now](#) | [View Webhook](#) | [View Subscriber](#) | [Open Failure](#)

Screen 8 — Retry & Dead-Letter Management

Purpose

Control automatic retry behavior and manage events that cannot be successfully delivered after the permitted retry attempts.

Retry Policy

Configure:

- • Maximum Attempts
- • Initial Delay
- • Retry Interval
- • Backoff Strategy
- • Maximum Delay
- • Retryable HTTP Codes
- • Non-Retryable Errors

Example

Attempt 1 — Immediate

Attempt 2 — +1 minute

Attempt 3 — +5 minutes

Attempt 4 — +30 minutes

Attempt 5 — +2 hours

Then:

Move to Dead-Letter Queue

Dead-Letter Queue

Event	Webhook	Failed Since	Attempts	Last Error	Status
-------	---------	--------------	----------	------------	--------

Failure Reasons

Examples:

- • Endpoint unavailable
- • Connection timeout
- • HTTP 500
- • HTTP 503
- • Invalid endpoint
- • Authentication rejected
- • Subscriber disabled

Bulk Actions

Authorized users may:

Retry Selected

Replay Selected

Move Back to Queue

Acknowledge / Close

Rules

Do not retry indefinitely.

Every retry attempt must remain traceable.

Screen 9 — Event Replay Center

Purpose

Allow authorized users to replay historical business events when a subscriber missed events, an integration was unavailable, or data synchronization needs to be recovered.

This is a critical enterprise integration capability.

Replay Search

Find events by:

- • Event ID
- • Event Type
- • Module
- • Tenant
- • Venue
- • Application
- • Subscriber
- • Date Range
- • Business Object ID
- • Correlation ID

Replay Selection

Support:

Single Event

or

Controlled Batch

Example:

Replay all ticket.sold events for Dubai Museum between 10:00 and 11:30.

Replay Preview

Before execution:

- **Events Found:** 4,821
- **Destination:** Museum ERP
- **Original Period:** 10:00–11:30
- **Estimated Replay Volume:** 4,821

Safety Controls

Require:

- • Destination validation
- • Authorization
- • Reason
- • Confirmation
- For large replays:
 - • • Approval may be required
 - • • Rate-controlled replay should be supported
 - • • Impact should be shown

Replay Metadata

The replayed delivery must preserve the original business-event identity while also identifying that the delivery is a replay.

Actions

Preview | Submit Replay | Approve | Start | Pause | Cancel

Rules

Replay must not create a new underlying business transaction.

For example:

- Replaying ticket.sold must **not sell the ticket again**.
- It only republishes the historical event.

Screen 10 — Event History, Retention & Audit

Purpose

Provide a complete historical record of TICVAI business events and webhook activity together with configurable retention and audit controls.

Event History

Search by:

- • Event ID
- • Event
- • Business Module
- • Business Object

- • Tenant
- • Venue
- • Subscriber
- • Application
- • Date
- • Status

Event Record

Show:

Event ID

Event Type

Version

Generated At

Source Module

Tenant

Business Object

Subscribers

Delivery Status

Replay Count

Event Timeline

Example:

- **10:02:14.120**
Ticket sold
- **10:02:14.140**
ticket.sold generated
- **10:02:14.160**
Matched 3 subscriptions
- **10:02:14.190**
Webhook A delivered
- **10:02:14.220**
Webhook B delivered
- **10:02:14.850**
Webhook C failed
- **10:03:14**
Webhook C retry successful

Retention Configuration

Support policies based on:

- • Event Category
- • Environment
- • Tenant
- • Regulatory/business requirements

- Example:
- • Operational Events: **90 days**
- • Financial Events: **7 years**, if required by the applicable business policy

The actual retention values must be configurable rather than hard-coded.

Audit

Track:

- • Webhook Created
- • Endpoint Changed
- • Secret Rotated
- • Subscription Created
- • Filter Modified
- • Webhook Suspended
- • Retry Triggered
- • Replay Requested
- • Replay Approved
- • Replay Executed

Actions

[View Event](#) | [View Deliveries](#) | [Replay](#) | [Export](#) | [View Audit](#)

Critical Event Envelope Standard

I recommend Soft Labs establish a **standard TICVAI event envelope** used across all business events.

Conceptually:

- {
- "eventId": "evt_123456",
- "eventType": "ticket.sold",
- "eventVersion": "1.0",
- "occurredAt": "2026-09-11T10:32:14Z",
- "tenantId": "tenant_001",
- "venueId": "venue_001",
- "source": "ticketing",
- "correlationId": "corr_789",
- "data": {
- "…": "business event payload"
- }
- }
- }
 - The actual schema will be finalized technically, but the principle is important:

Every TICVAI event should have a consistent envelope around its module-specific payload.

This makes integrations significantly easier to maintain.

Event Delivery Reliability

I recommend Board 5 explicitly target **at-least-once delivery semantics** unless Soft Labs proposes and documents another architecture.

That means a subscriber could occasionally receive the same event more than once.

Therefore each event needs a stable:

eventId

so external systems can implement idempotency/deduplication.

For example:

- Partner receives:
- • evt_123456
- • twice.

Their integration recognizes the same Event ID and does not process the ticket sale twice.

This is especially important for payments, wallet transactions, refunds, ticket issuance and other financial/transactional events.

Webhook Health Model

Each webhook should have a health status:

Healthy

Normal delivery success and latency.

Warning

Failure/latency approaching configured threshold.

Degraded

Repeated failures or slow acknowledgment.

Critical

Large failure rate or sustained unavailability.

Suspended

TICVAI automatically or manually stopped delivery.

This status should be visible in Screens 1, 4 and 7.

Cross-Board Boundaries

Function	Owner
API credentials	Board 2
API access/scopes/security	Board 3
API performance/error analytics	Board 4
Business event catalog	Board 5
Webhook endpoints	Board 5
Event subscriptions	Board 5
Event routing/filtering	Board 5

Function	Owner
Webhook delivery	Board 5
Retry/dead-letter	Board 5
Historical event replay	Board 5
Event retention/history	Board 5
API/event version lifecycle	Board 6
Sandbox webhook testing environment	Board 7
Business transaction itself	Owning TICVAI module

The final point is especially important:

Board 5 distributes events. It does not own the business transaction.

A refund remains owned by Finance/Payments/Ticketing. Board 5 simply receives:

- ticket.refunded
- and distributes that event to authorized subscribers.

Board 5 — Final Screen Map

Webhooks & Event Management

1. Event & Webhook Command Center
2. Business Event Catalog
3. Webhook Registration
4. Webhook Configuration
5. Event Subscription Management
6. Event Routing & Filtering
7. Webhook Delivery Monitor
8. Retry & Dead-Letter Management
9. Event Replay Center
10. Event History, Retention & Audit

This gives TICVAI a proper **enterprise event-driven integration layer**, rather than treating webhooks simply as URLs attached to APIs.

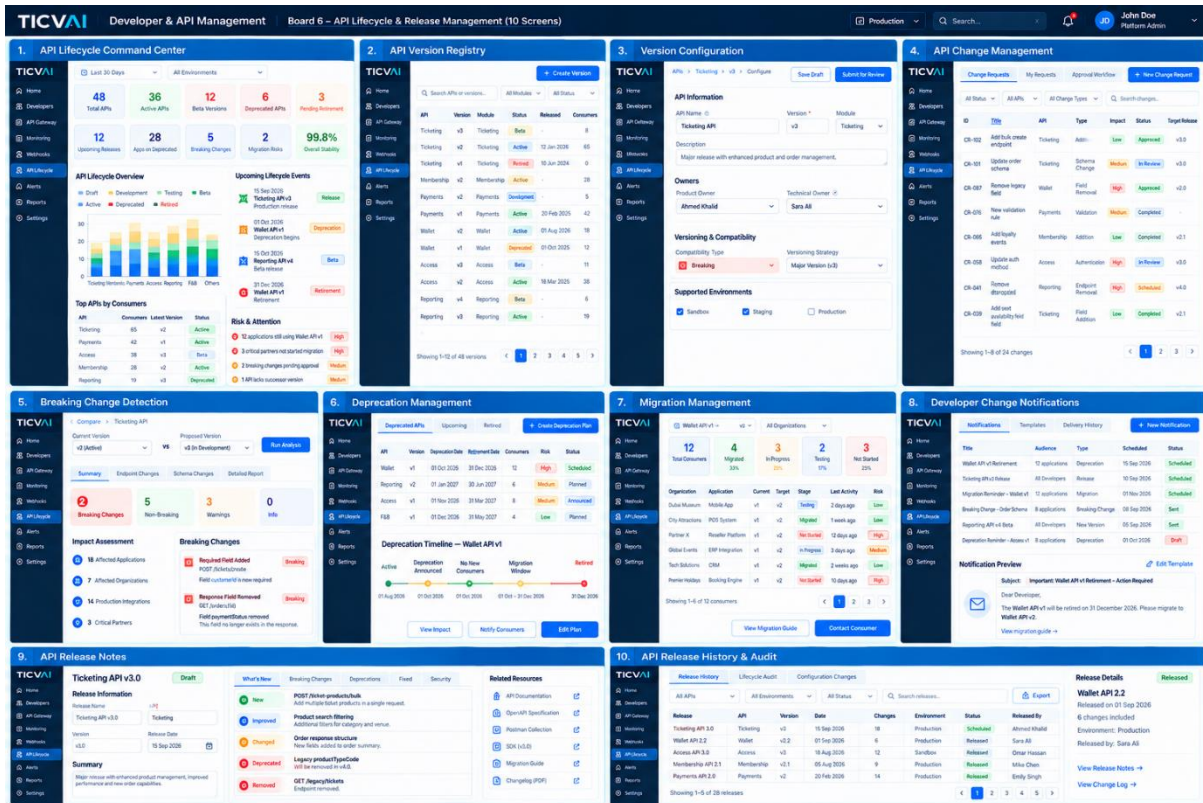
Board 6 — API Lifecycle & Release Management

Module: 13 — Developer & API Management

Board: 6 of 8

Screens: 10

Board Type: API Governance / Versioning / Change Management / Deprecation / Release Control



Board Purpose

Provide TICVAI with a centralized governance layer for managing the **complete lifecycle of APIs**, from initial development and version creation through release, change management, migration, deprecation, and retirement.

Board 6 ensures that TICVAI can continuously evolve its APIs **without unexpectedly breaking client, partner, reseller, mobile-app, POS, or third-party integrations**.

The board should answer:

What API versions currently exist?

Which version is production?

What changed between versions?

Will this change break an existing integration?

Which applications are still using an old version?

When will an API be deprecated?

Which customers need to migrate?

Have developers been notified?

What exactly was released?

The lifecycle should follow:

Draft → Development → Testing → Beta → Active → Deprecated → Retired

And for a major change:

Change Proposed → Impact Analysis → Breaking-Change Check → Approval → Release → Migration Window → Deprecation → Retirement

Screen 1 — API Lifecycle Command Center

Purpose

Provide management and technical teams with one central view of the lifecycle status of all TICVAI APIs and API versions.

Header KPIs

- Total APIs
- Active APIs
- Active Versions
- Beta Versions
- Deprecated APIs
- APIs Pending Retirement
- Upcoming Releases
- Applications Using Deprecated Versions
- Breaking Changes Pending
- Migration Risks

Filters

- Business Module
- API
- Version
- Lifecycle Status
- Environment
- Release
- Owner
- Planned Retirement Date

Lifecycle Overview

Visualize APIs by stage:

Draft → Development → Testing → Beta → Active → Deprecated → Retired

Example:

API	Current	Next	Status	Consumers	Action
Ticketing	v2	v3	Active	65	v3 Beta
Membership	v2	—	Active	28	—
Wallet	v1	v2	Deprecated	12	Migration
Reporting	v3	v4	Beta	19	Testing

Upcoming Lifecycle Events

Examples:

15 Sep — Ticketing API v3 production release

01 Oct — Wallet API v1 deprecation begins

31 Dec — Wallet API v1 retirement

Risk Panel

Show:

- 12 applications remain on Wallet API v1.
- 3 critical partners have not started migration.

Actions

Create Version | Propose Change | View Migration | Create Release | Review Deprecations

Acceptance Criteria

The Command Center must provide lifecycle visibility without duplicating operational API monitoring from Board 4.

Screen 2 — API Version Registry

Purpose

Maintain the authoritative registry of all API versions across TICVAI.

UI / Configuration

API Version Directory

API	Version	Status	Released	Consumers	Previous	Successor
Ticketing	v3	Beta	—	8	v2	—
Ticketing	v2	Active	Jan 2026	65	v1	v3
Ticketing	v1	Retired	Jun 2024	0	—	v2

Version Status

Support:

- Draft
- Development
- Testing
- Beta
- Active
- Deprecated
- Retired

Version Detail

Show:

- API

- Version
- Business Module
- Owner
- Created Date
- Release Date
- Lifecycle Status
- Supported Environments
- Predecessor
- Successor
- Consumers
- Documentation
- OpenAPI Specification
- Release
- Planned Deprecation
- Planned Retirement

Version Relationships

Visual representation:

v1 Retired → v2 Active → v3 Beta

Actions

[Create Version](#) | [Open Version](#) | [Compare](#) | [View Consumers](#) | [View Documentation](#)

Rules

- Versions require unique identifiers.
- Retired versions cannot become active through an ordinary status change.
- Lifecycle transitions must be audited.

Screen 3 — Version Configuration

Purpose

Configure the governance and technical metadata of an individual API version.

General Configuration

- API Name
- Version
- Version Description
- Business Module
- Product Owner
- Technical Owner
- Environment Availability

Versioning Strategy

Support TICVAI's approved versioning model, for example:

- **Major Version**

v1 → v2

- Used for breaking changes.

Minor / Compatible Release

Where supported by TICVAI API standards.

Compatibility

Set:

- • Backward Compatible
- • Potentially Breaking
- • Breaking
- • Experimental

Lifecycle Dates

Configure:

- • Development Start
- • Beta Date
- • Production Release
- • Deprecation Date
- • End-of-Support Date
- • Retirement Date

API Definition

Associate:

- • OpenAPI Specification
- • Schema
- • Endpoint Set
- • Event Versions where relevant
- • Documentation Version

Consumer Policy

Configure:

- • New Consumers Allowed
- • Existing Consumers Only
- • Migration Required
- • No New Consumers

Actions

Save Draft | Validate | Submit for Review | Publish Beta | Activate

Acceptance Criteria

A version must not be activated without the required documentation and technical definition.

Screen 4 — API Change Management

Purpose

Provide a governed process for proposing and managing changes to existing APIs.

Change Request

Fields:

- • Change Title
- • API
- • Current Version
- • Target Version
- • Change Type
- • Description
- • Business Reason
- • Requested By
- • Target Release

Change Types

Support:

- • Add Endpoint
- • Modify Endpoint
- • Remove Endpoint
- • Add Field
- • Modify Field
- • Remove Field
- • Change Validation
- • Change Authentication
- • Change Scope
- • Change Response
- • Change Error Behavior
- • Schema Change
- • Event Change

Change Classification

Non-Breaking

Example:

Optional field added.

Potentially Breaking

Example:

Validation behavior changed.

Breaking

Example:

Required field introduced.

Impact Summary

Show:

- • Endpoints affected
- • Schemas affected
- • Applications potentially affected
- • Organizations affected

- SDKs affected
- Postman collections affected
- Documentation affected

Workflow

Draft → Technical Review → Impact Review → Approval → Scheduled → Released

Actions

Create Change | Analyze Impact | Submit | Approve | Reject | Schedule

Screen 5 — Breaking Change Detection

Purpose

Automatically identify potentially breaking changes before an API version or release is published.

This is one of the most important controls on Board 6.

Comparison

Select:

- **Current Definition:** Ticketing API v2

vs.

- **Proposed Definition:** Ticketing API v3

Automatic Detection

Detect changes such as:

- Endpoint removed
- HTTP method changed
- Required parameter added
- Parameter removed
- Data type changed
- Required field introduced
- Response field removed
- Enum value changed
- Authentication changed
- Scope changed
- Error behavior changed

Results

Example:

2 Critical Breaking Changes

 /tickets/create

customerId

Before: Optional

After: Required

 /orders/{id}

Response field:

- paymentStatus
- Removed

Impacted Consumers

Show:

Applications affected: 18

Organizations affected: 7

Production integrations: 14

Risk Level

- Low
- Medium
- High
- Critical

Actions

[View Difference](#) | [View Consumers](#) | [Accept as Breaking](#) | [Modify Change](#) | [Create Migration Requirement](#)

Rule

Critical breaking changes must not be released as an ordinary backward-compatible update.

Screen 6 — Deprecation Management

Purpose

Govern the controlled withdrawal of old APIs, endpoints, versions, schemas, and event versions.

Deprecation Directory

API	Version	Deprecation Date	Retirement	Consumers	Risk
Wallet	v1	01 Oct	31 Dec	12	High
Ticketing	v1	Completed	30 Jun	0	Closed

Create Deprecation Plan

Configure:

- API / Version
- Deprecation Date
- End-of-Support Date
- Retirement Date
- Replacement Version
- Migration Guide
- Notification Schedule
- New Consumer Restriction

Deprecation Stages

Active

↓

Deprecation Announced

↓

No New Consumers

↓

Migration Window

↓

End of Support

↓

Retired

Consumer Impact

Show:

- 12 applications currently use Wallet v1.
- Break down:

- **Production:** 9

Sandbox: 3

Critical Partners: 2

Actions

[Start Deprecation](#) | [Extend Deadline](#) | [Notify Consumers](#) | [View Migration](#) | [Retire](#)

Rules

Retirement must be blocked or require elevated approval when critical consumers remain, according to TICVAI governance policy.

Screen 7 — Migration Management

Purpose

Track the migration of developers, clients, and partner integrations from one API version to another.

Migration Campaign

Example:

Wallet API v1 → v2

Header KPIs

- Total Consumers
- Migrated
- In Progress
- Not Started
- Testing

- Blocked
- Production Complete

Consumer Table

Organization	Application	Current	Target	Stage	Last Activity	Risk
Dubai Museum	Mobile App	v1	v2	Testing	Today	Low
Partner X	ERP	v1	v2	Not Started	12 days	High

Migration Stages

Not Started

- Development
- Sandbox Testing
- Certification where required
- Production Ready
- Production Migrated
- Verified

Migration Guide

Link:

- Breaking Changes
- New Fields
- Removed Fields
- Authentication Changes
- Code Samples
- SDK Upgrade
- Postman Collection
- Test Cases

Migration Readiness

Example:

Dubai Museum Mobile App

- ✓ v2 credentials available
- ✓ Sandbox calls successful
- ✓ Required endpoints tested
- ⚠ Production cutover not scheduled

Actions

[Start Migration](#) | [Update Stage](#) | [Contact Consumer](#) | [View Testing](#) | [Mark Complete](#)

Acceptance Criteria

TICVAI must be able to identify exactly which applications remain dependent on an API scheduled for retirement.

Screen 8 — Developer Change Notifications

Purpose

Manage communication with developers and organizations affected by API changes, releases, deprecations, and migrations.

Notification Types

- New API Version
- API Change
- Breaking Change
- Deprecation Notice
- Retirement Reminder
- Migration Reminder
- Release Announcement
- Security-Related API Change

Audience Selection

Target by:

- • API Consumers
- • API Version
- • Organization
- • Application
- • Developer
- • Partner Type
- • Environment

Example

Subject

Wallet API v1 Retirement — Action Required

Audience

12 active applications using Wallet v1

Replacement

Wallet API v2

Retirement

31 December 2026

Notification Schedule

Example:

90 days before

Initial notice

60 days

Migration reminder

30 days

Urgent reminder

7 days

Final warning

Delivery Channels

According to TICVAI's supported notification architecture:

- • Developer Portal
- • Email
- • In-platform Notification
- • Other approved channels

Delivery Status

Show:

- • Sent
- • Delivered
- • Viewed/Acknowledged where supported
- • Failed

Actions

[Create Notification](#) | [Preview](#) | [Schedule](#) | [Send](#) | [Resend](#)

Screen 9 — API Release Notes**Purpose**

Create and publish clear developer-facing release notes describing what changed in a TICVAI API release.

Release Note Header

- Release Name
- API
- Version
- Release Date
- Status

Change Categories**New**

New endpoints/features.

Improved

Backward-compatible improvements.

Changed

Behavior changes.

Deprecated

Functionality scheduled for retirement.

Removed

Removed functionality.

Fixed

API defects resolved.

Security

Relevant security-related changes.

Example

Ticketing API v3

New

POST /ticket-products/bulk

Changed

Product search now supports additional filtering.

Deprecated

Legacy productTypeCode.

Migration Required

Yes — for applications using legacy product codes.

Related Resources

- Documentation
- Migration Guide
- OpenAPI
- SDK
- Postman Collection
- Code Samples

Actions

Create | Preview | Publish | Update | View Developer Portal

Rule

Release notes must correspond with an approved release/version and should not become an independent source of undocumented API changes.

Screen 10 — API Release History & Audit

Purpose

Provide a complete immutable record of API releases and lifecycle actions.

Release History

Release	API	Version	Date	Changes	Owner	Status
Ticketing 3.0	Ticketing	v3	15 Sep	18	API Team	Released

Release	API	Version	Date	Changes	Owner	Status
Wallet 2.2	Wallet	v2	01 Sep	6	Wallet Team	Released

Release Detail

Show:

- Release ID
- APIs Included
- Versions
- Environment
- Release Date/Time
- Approved By
- Deployed By
- Change Requests
- Breaking Changes
- Documentation
- Release Notes
- Migration Requirements

Lifecycle Audit

Track:

Version Created

Change Proposed

Change Approved

Breaking Change Accepted

Beta Published

Production Activated

Deprecation Announced

Retirement Extended

API Retired

Before / After

For configuration changes show:

Before

Wallet v1 Retirement:
31 Dec 2026

After

31 Mar 2027

Reason

Critical partner migration extension.

Audit Fields

- Timestamp
- User
- Action
- API
- Version
- Previous Value
- New Value
- Reason
- Approval Reference

Actions

[View Release](#) | [View Changes](#) | [Compare](#) | [Export Audit](#) | [View Approval](#)

Critical Lifecycle Governance

Soft Labs should implement controlled state transitions.

An API version should not simply have a dropdown allowing administrators to jump from:

Draft → Active

Instead, lifecycle gates should apply.

For example:

Draft → Testing

Requires:

- ✓ API definition available
- ✓ Schema validation passed

Testing → Beta

Requires:

- ✓ Documentation available
- ✓ Test environment available
- ✓ Security review completed

Beta → Active

Requires:

- ✓ Testing completed
- ✓ Breaking-change analysis completed
- ✓ Required approvals completed
- ✓ Documentation published
- ✓ SDK/Postman updates where applicable

Active → Deprecated

Requires:

- ✓ Replacement identified where applicable
- ✓ Migration plan

- ✓ Consumer impact analysis
- ✓ Notification plan

Deprecated → Retired

Requires:

- ✓ Migration window completed
- ✓ Remaining consumers reviewed
- ✓ Required approval
- ✓ Retirement communication completed

This makes the API lifecycle governed rather than simply status-driven.

API Compatibility Principle

Board 6 should enforce a simple architectural rule:

Non-Breaking Change

May generally remain within the compatible version/release according to TICVAI's versioning policy.

Examples:

- • Add optional response field
- • Add new endpoint
- • Add optional request parameter

Breaking Change

Should normally require a new major API version.

Examples:

- • Remove endpoint
- • Remove response field
- • Change field type
- • Make optional field required
- • Change authentication behavior
- • Fundamentally change response structure

The final versioning policy can be technically agreed with Soft Labs, but the platform must support this governance model.

Relationship with Board 1

This is important.

Board 6 manages:

API version/change/release

After publication, Board 1 automatically reflects:

Documentation

Endpoint version

OpenAPI specification

Code samples

SDK compatibility

Postman collection

Release notes

Therefore developers should never find:

- Documentation says v3
- while:
 - SDK only supports v2
 - without a visible compatibility warning.

Board 6 is effectively the **source of lifecycle truth**, while Board 1 is the developer-facing documentation experience.

Cross-Board Boundaries

Function	Owner
API documentation	Board 1
Developer accounts/credentials	Board 2
API security/access	Board 3
API performance/usage	Board 4
Event/webhook operation	Board 5
API versions	Board 6
API changes	Board 6
Breaking-change detection	Board 6
Deprecation	Board 6
Migration management	Board 6
Developer change notifications	Board 6
Release notes/history	Board 6
Sandbox environments/testing	Board 7
Partner certification	Board 8

Board 6 — Final Screen Map

API Lifecycle & Release Management

1. API Lifecycle Command Center
2. API Version Registry
3. Version Configuration
4. API Change Management
5. Breaking Change Detection
6. Deprecation Management
7. Migration Management
8. Developer Change Notifications
9. API Release Notes
10. API Release History & Audit

This board gives TICVAI a controlled way to **evolve APIs for years without losing governance or unexpectedly breaking customer integrations.**

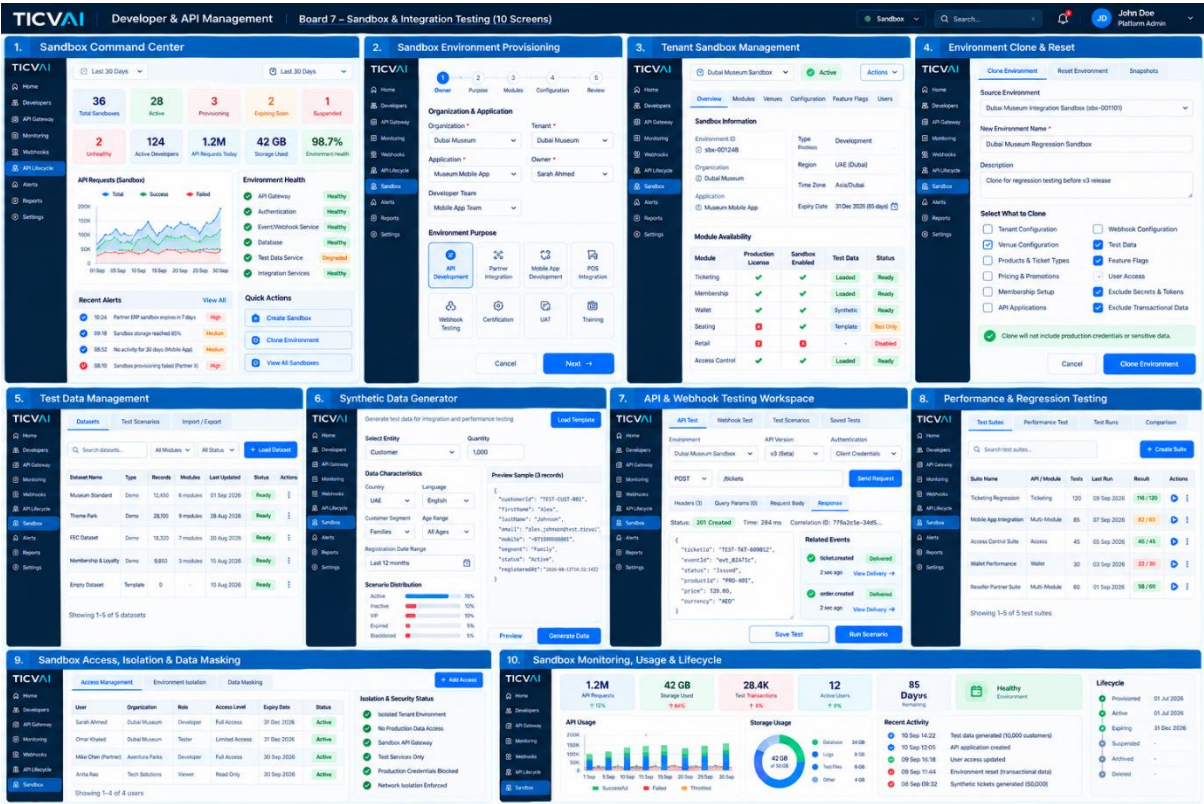
Board 7 — Sandbox & Integration Testing

Module: 13 — Developer & API Management

Board: 7 of 8

Screens: 10

Board Type: Sandbox Administration / Test Data / Integration Testing / Environment Governance



Board Purpose

Provide TICVAI with a centralized **sandbox and integration-testing environment** where clients, developers, partners, resellers, and internal teams can safely build and validate integrations without affecting live production data or operations.

Board 7 shall support the full sandbox lifecycle:

Request / Provision → Configure → Populate Test Data → Develop → Test APIs & Webhooks → Run Regression/Performance Tests → Monitor Usage → Reset/Clone → Expire or Extend

The sandbox must support **multi-tenant isolation**, controlled access, synthetic or masked data, API testing, webhook testing, environment cloning/reset, monitoring, audit, usage analytics, and configurable expiration.

A sandbox should behave sufficiently like TICVAI production for realistic integration testing, while remaining completely isolated from live production transactions.

Screen 1 — Sandbox Command Center

Purpose

Provide administrators with a centralized overview of all TICVAI sandbox environments, their status, usage, health, ownership, and lifecycle.

Header KPIs

- Total Sandboxes
- Active Sandboxes

- Provisioning
- Expiring Soon
- Suspended
- Failed / Unhealthy
- Active Developers
- API Requests Today
- Storage Consumed
- Test Runs Today

Filters

- Tenant
- Organization
- Application
- Sandbox Type
- Status
- Owner
- Region
- Expiration
- Created Date

Sandbox Directory

Sandbox	Organization	Tenant	Owner	Created	Expiry	Usage	Health	Status
---------	--------------	--------	-------	---------	--------	-------	--------	--------

Environment Health

Show:

- • API Gateway
- • Authentication
- • Event/Webhook Service
- • Database
- • Test Data
- • Integration Services
- • Background Jobs

Expiration Alerts

Examples:

- Partner A Sandbox expires in 7 days.
- Museum Mobile Sandbox has been inactive for 30 days.
- Sandbox storage has reached 85% of quota.

Quick Actions

Create Sandbox | Clone | Reset | Extend | Suspend | Open Environment

Acceptance Criteria

Board 7 must clearly distinguish **Sandbox**, **Staging/UAT where supported**, and **Production** so users cannot accidentally believe they are operating against production.

Screen 2 — Sandbox Environment Provisioning

Purpose

Create a new isolated TICVAI sandbox environment using a governed provisioning workflow.

Provisioning Wizard

Step 1 — Owner

Select:

- Organization
- Tenant
- Application
- Sandbox Owner
- Developer Team

Step 2 — Environment Purpose

Examples:

- API Development
- Partner Integration
- Mobile App Development
- POS Integration
- Access Control Integration
- Webhook Testing
- Certification
- UAT
- Training

Step 3 — TICVAI Modules

Select modules available in the sandbox:

- Ticketing
- Pricing
- Promotions
- Membership
- Loyalty
- Wallet
- CRM
- Reservations
- Seating
- Access Control
- F&B
- Retail
- Reporting

- • AI
- • Other licensed modules

Environment Configuration

- Sandbox Name
- Region
- Environment Type
- Default Currency
- Time Zone
- Language
- Expiration Date
- Data Template

Capacity / Quotas

Configure:

- • API Request Allowance
- • Storage
- • Test Customers
- • Test Products
- • Test Transactions
- • Webhook Deliveries
- • Concurrent Users

Provisioning Preview

Before creation:

Environment: Dubai Museum Mobile Sandbox

Modules: 8

Users: 6

Data Template: Museum Demo

Duration: 90 Days

Estimated Provisioning: 4 minutes

Actions

[Provision](#) | [Save Template](#) | [Cancel](#)

Provisioning Status

Show stages:

Requested → Allocating → Configuring → Loading Data → Validating → Ready

Rules

The sandbox must not connect to production databases or production transactional services unless a specifically governed read-only mechanism is architecturally approved.

Screen 3 — Tenant Sandbox Management

Purpose

Manage sandbox environments in TICVAI's multi-tenant architecture and control what each sandbox is permitted to simulate.

Tenant Sandbox Header

Show:

- • Sandbox
- • Organization
- • Tenant
- • Application
- • Owner
- • Environment ID
- • Status
- • Expiry

Tenant Configuration

Configure:

- • Tenant Context
- • Venue(s)
- • Business Units
- • Modules
- • Currency
- • Time Zone
- • Locale
- • Feature Flags

Module Availability

Example:

Module	Production License	Sandbox Enabled	Test Data	Status
Ticketing	✓	✓	Loaded	Ready
Membership	✓	✓	Loaded	Ready
Wallet	✓	✓	Synthetic	Ready
Seating	✗	✓ Demo	Template	Test Only
Retail	✗	✗	—	Disabled

Where TICVAI commercial policy permits, certain modules may be made available for sandbox evaluation even when not licensed in production.

Venue Simulation

Allow:

- • Existing sandbox venue
- • Demo venue

- • Multiple test venues

Environment Features

Configure:

- • Feature Flags
- • Experimental APIs
- • Beta APIs
- • Test Integrations
- • Mock Services

Actions

[Edit Environment](#) | [Add Venue](#) | [Enable Test Module](#) | [Manage Features](#) | [Suspend](#)

Rules

Sandbox tenant boundaries must be enforced independently from production tenant boundaries.

Screen 4 — Environment Clone & Reset

Purpose

Allow authorized users to safely clone an existing test configuration or return a sandbox to a known clean state.

Clone Environment

Source:

Dubai Museum Integration Sandbox

Clone into:

Dubai Museum Regression Sandbox

Clone Options

Select what to copy:

- • Tenant Configuration
- • Venue Configuration
- • Products
- • Pricing
- • Promotions
- • Membership Setup
- • Access Rules
- • Seating
- • API Applications
- • Webhook Configuration
- • Test Data
- • Feature Flags

Exclusions

Sensitive items should not automatically clone where prohibited:

- • Secrets

- Tokens
- Sensitive personal data
- Production credentials

Reset Environment

Reset options:

Configuration Only

Transactional Data Only

Test Data Only

Complete Environment Reset

Reset Preview

Before reset:

Products retained: 24

Test Orders deleted: 8,420

Test Customers deleted: 2,800

Webhook history cleared: 12,300

API configuration retained.

Restore Point

Where supported, allow:

Create Snapshot Before Reset

Actions

Clone | Create Snapshot | Reset | Restore Snapshot

Rules

Reset and clone actions must be audited and require confirmation because they may delete significant test data.

Screen 5 — Test Data Management

Purpose

Provide centralized management of the data developers require to test TICVAI integrations.

Data Categories

- Venues
- Products
- Ticket Types
- Pricing
- Promotions
- Customers
- Memberships
- Loyalty Accounts
- Wallets

- Reservations
- Orders
- Tickets
- Payments
- Access Events
- Resources
- Seats
- F&B/Retail Products

Test Data Sets

Examples:

Museum Standard Dataset

Theme Park Dataset

FEC Dataset

Theatre / Assigned Seating Dataset

Membership & Loyalty Dataset

Dataset Information

Show:

- • Dataset Name
- • Type
- • Records
- • Modules
- • Created By
- • Last Updated
- • Data Source
- • Status

Data Actions

Support:

- • Load Dataset
- • Clear Dataset
- • Refresh Dataset
- • Export
- • Import approved test data
- • Duplicate Dataset

Test Scenarios

Example:

Sold-Out Timeslot

Creates:

- • Product
- • Schedule

- • Capacity
- • Reservations
- • Tickets
- so the developer can test sold-out behavior.
- Other scenarios:
 - • • Expired Membership
 - • • Insufficient Wallet Balance
 - • • Refund
 - • • Cancelled Event
 - • • Invalid Ticket
 - • • Duplicate Scan
 - • • Fully Booked Session

Actions

[Load Dataset](#) | [Create Dataset](#) | [Import](#) | [Export](#) | [Clear Data](#)

Screen 6 — Synthetic Data Generator

Purpose

Generate realistic but non-production data for integration, performance, regression, and workflow testing.

Data Generation Wizard

Select entity:

- • Customer
- • Ticket
- • Order
- • Reservation
- • Membership
- • Wallet
- • Payment
- • Access Event
- • Product
- • Event
- • Resource

Volume

Examples:

100 Customers

10,000 Tickets

1,000,000 Access Events

Data Characteristics

Configure where relevant:

- • Country
- • Language

- • Customer Segment
- • Age Range
- • Product Mix
- • Transaction Value
- • Date Range
- • Channel
- • Membership Tier
- • Payment Status
- • Ticket Status

Scenario Distribution

Example:

Tickets:

Valid — 70%

Used — 15%

Expired — 5%

Refunded — 5%

Cancelled — 3%

Invalid — 2%

Preview

Show sample generated records before execution.

Actions

Generate | Preview | Save Template | Schedule Generation | Delete Generated Data

Rules

Synthetic data must be clearly identifiable as non-production data.

Generated customer identities must not unintentionally correspond to actual customer records.

Screen 7 — API & Webhook Testing Workspace

Purpose

Provide one integrated workspace where developers can test APIs and event/webhook behavior against their sandbox.

This screen complements the simple interactive API Explorer from Board 1 by providing a more complete **integration-testing workspace**.

Workspace Header

Select:

- • Sandbox
- • Application
- • API Version
- • Test Scenario

API Test

Configure:

- • API
- • Endpoint
- • Method
- • Headers
- • Parameters
- • Request Body

Execute

Send Request

Show:

- • Status Code
- • Response
- • Duration
- • Correlation ID
- • Logs
- • Created Business Object where applicable

Webhook Test

Allow developer to:

1. • Execute business operation.
2. • Confirm event generated.
3. • Confirm subscription matched.
4. • Confirm webhook delivery.
5. • Inspect payload.
6. • Inspect subscriber response.

• Example:

- • POST /tickets

↓

- • Ticket Created

↓

- • ticket.created

↓

- • CRM Webhook

↓

200 Delivered

Assertions

Support:

Expected HTTP = 201

Response contains ticketId

Webhook generated = ticket.created

Delivery status = successful

Test Results

Passed / Failed

Actions

[Run](#) | [Save Test](#) | [Re-run](#) | [Compare](#) | [Export Result](#)

Screen 8 — Performance & Regression Testing

Purpose

Allow TICVAI and authorized integration teams to validate that integrations continue working correctly and perform adequately.

Test Suite Directory

Examples:

- Ticketing Regression Suite
- Mobile App Integration Suite
- Partner Reseller Suite
- Wallet Integration Suite
- Access Validation Performance Suite

Regression Tests

Each suite contains test cases such as:

Create Ticket

Retrieve Ticket

Refund Ticket

Validate Ticket

Receive Webhook

Test Run

Show:

120 Tests

✓ Passed: 116

✗ Failed: 3

⚠ Warning: 1

Version Comparison

Example:

Ticketing API v2

120/120 Passed

vs.

Ticketing API v3 Beta

117/120 Passed

Highlight failures.

Performance Testing

Configure:

- • Requests per second
- • Concurrent Users
- • Duration
- • Ramp-Up
- • API/Endpoint
- • Dataset

Results

Show:

- • Throughput
- • Average Latency
- • P95
- • P99
- • Error Rate
- • Success Rate

Safety

Performance tests must run only against environments specifically approved for load testing.

Actions

[Create Suite](#) | [Run Tests](#) | [Compare Version](#) | [View Failure](#) | [Export Report](#)

Screen 9 — Sandbox Access, Isolation & Data Masking

Purpose

Control who can access a sandbox and guarantee that test environments remain properly isolated and privacy-safe.

Access Management

Show:

- • Developer
- • Organization
- • Team
- • Role
- • Sandbox
- • Access Level

- • Expiry

Access Levels

Examples:

- • Sandbox Administrator
- • Developer
- • Tester
- • Viewer

Environment Isolation

Display validation status:

- ✓ Dedicated Tenant Context
- ✓ Production Database Access Blocked
- ✓ Production Credentials Blocked
- ✓ Sandbox API Gateway
- ✓ Sandbox Event Bus
- ✓ Sandbox Storage

Data Masking

For approved scenarios where data is derived from a protected source, configure masking for fields such as:

- • Name
- • Email
- • Mobile
- • Address
- • Customer ID
- • Payment Reference
- • Membership Identifier
- Example:
 - • Original:
 - • • john.smith@example.com
 - • • Masked:
 - • • user_82741@test.ticvai

Data Protection Rules

Support:

- • Mask
- • Tokenize where supported
- • Remove
- • Replace with Synthetic
- • Block Field

Actions

Grant Access | Revoke | Extend | Configure Masking | Validate Isolation

Acceptance Criteria

Production-sensitive data must never be copied into sandbox in readable form merely for convenience.

Screen 10 — Sandbox Monitoring, Usage & Lifecycle

Purpose

Manage sandbox consumption, audit activity, expiration, renewal, suspension, and deletion throughout the environment lifecycle.

Header KPIs

- API Requests
- Storage Used
- Test Transactions
- Active Users
- Webhook Deliveries
- Days Remaining

Usage vs Quota

Example:

API Requests

820K / 1M — 82%

Storage

42 GB / 50 GB — 84%

Test Customers

7,200 / 10,000 — 72%

Activity

Show:

- • Last Login
- • Last API Call
- • Last Test Run
- • Last Configuration Change
- • Last Data Generation
- • Last Reset

Lifecycle

Visualize:

Provisioned → Active → Expiring → Extended / Expired → Archived → Deleted

Expiration Policy

Configure:

- • Expiration Date
- • Warning Period
- • Auto-Suspend
- • Archive Period
- • Auto-Delete
- Example:

- • **14 days before expiry**

Notify owner.

- • **Expiry date**

Suspend.

- • **30 days after expiry**

Delete unless extended.

Renewal / Extension

Require:

- • Requested Extension
- • Reason
- • New Expiry
- • Approval where required

Audit History

Track:

- • Created
- • Provisioned
- • Module Enabled
- • Developer Added
- • Dataset Loaded
- • Reset
- • Clone
- • Extension
- • Suspension
- • Deletion

Actions

[Extend](#) | [Suspend](#) | [Reactivate](#) | [Archive](#) | [Delete](#) | [Export Usage](#) | [View Audit](#)

Critical Sandbox Architecture

For Soft Labs, the following principle should be explicit:

Sandbox must be operationally isolated from Production.

A sandbox API request should conceptually follow:

Developer Application

↓

Sandbox Credential

↓

Sandbox API Gateway

↓

Sandbox Tenant

↓

Sandbox Business Services / Approved Test Services

↓

Sandbox Database / Test Dataand **not**:**Sandbox → Production Customer Data**

unless TICVAI explicitly designs a controlled and authorized mechanism for a specific purpose.

Sandbox Identity

Every sandbox resource should be clearly marked.

For example:

Environment**SANDBOX****Ticket**

TEST-TKT-009812

Customer

TEST-CUST-00142

Payment**SIMULATED**

This prevents test transactions from being confused with real venue transactions.

Simulated External ServicesI also recommend that the sandbox support **mock/simulated external services** where practical.

For example:

Payment

Developer selects:

Simulate Payment Result

- Approved
- Declined
- Timeout
- 3DS Required
- Refund Successful
- Refund Failed

Access Control

Simulate:

- • Valid Scan
- • Invalid Scan
- • Duplicate Scan

- • Offline Scan
- • Anti-Passback Failure

Webhook

Simulate:

- • HTTP 200
- • HTTP 400
- • HTTP 500
- • Timeout

This will make Board 7 significantly more useful for TICVAI implementations.

Relationship with Board 6

Board 7 becomes especially important when TICVAI releases a new API version.

For example:

Board 6

Ticketing API v3 → Beta

↓

Board 7

Enable v3 in Partner Sandbox

↓

Partner runs regression tests

↓

117 / 120 Passed

↓

Fix integration

↓

120 / 120 Passed

↓

Partner marked migration-ready

↓

Board 6

Production migration

This creates a very strong lifecycle between **API governance and actual integration testing**.

Cross-Board Boundaries

Function	Owner
Developer documentation/API Explorer	Board 1
Credentials	Board 2

Function	Owner
API security/scopes	Board 3
Production API monitoring	Board 4
Webhook/event management	Board 5
API version/migration lifecycle	Board 6
Sandbox provisioning	Board 7
Sandbox tenant management	Board 7
Clone/reset	Board 7
Test/synthetic data	Board 7
API & webhook integration testing	Board 7
Regression/performance testing	Board 7
Sandbox isolation/masking	Board 7
Sandbox usage/lifecycle	Board 7
Partner certification	Board 8

Board 7 — Final Screen Map

Sandbox & Integration Testing

1. **Sandbox Command Center**
2. **Sandbox Environment Provisioning**
3. **Tenant Sandbox Management**
4. **Environment Clone & Reset**
5. **Test Data Management**
6. **Synthetic Data Generator**
7. **API & Webhook Testing Workspace**
8. **Performance & Regression Testing**
9. **Sandbox Access, Isolation & Data Masking**
10. **Sandbox Monitoring, Usage & Lifecycle**

This gives TICVAI a complete **safe integration laboratory** where a partner can go from an empty environment all the way through development, API/webhook testing, regression validation, and ultimately production readiness without touching live operations.

Board 8 — Business API Coverage & Partner Ecosystem

Module: 13 — Developer & API Management

Board: 8 of 8

Screens: 10

Board Type: Business API Coverage / Partner Integration / Certification / Marketplace



Board Purpose

Provide TICVAI with the governance layer that ensures **business functionality available through the TICVAI administration platform is appropriately exposed through documented APIs**, while also providing the ecosystem required for partners, system integrators, resellers, technology providers, and clients to build, certify, publish, and maintain integrations.

This board addresses two major objectives:

1. API Coverage Governance

TICVAI should be able to answer:

- Which business functions are available through APIs?
- Which UI capabilities are not yet exposed?
- Which modules support Read/Create/Update/Delete/Execute operations?
- Which business events are available?
- Are there gaps between the TICVAI UI and TICVAI APIs?

2. Partner Ecosystem

TICVAI should provide a structured environment where approved partners can:

Register integrations → Develop → Test → Submit for certification → Resolve findings → Receive approval → Publish to the Integration Marketplace.

The governing principle should be:

All appropriate TICVAI business functionality shall be exposed through documented APIs unless explicitly restricted for security, regulatory, architectural, or operational reasons.

Screen 1 — Business API Coverage Matrix

Purpose

Provide a centralized governance view showing the API coverage of every TICVAI business module and identify functionality that exists in the platform but is not yet exposed through APIs.

This should be the **master screen for requirement 13.3**.

Header KPIs

- Total Business Modules
- Business Functions
- API-Enabled Functions
- API Coverage %
- Event-Enabled Functions
- Coverage Gaps
- Restricted Functions
- APIs Pending Development

Filters

- Business Module
- Sub-Module
- Business Function
- API Status
- API Version
- Environment
- Coverage Status
- Event Availability
- Restriction Type

Coverage Matrix

The main interface should use a matrix rather than creating hundreds of separate screens.

Business Function	Read	Create	Update	Delete	Execute	Events	Coverage
Ticket Product	✓	✓	✓	—	✓	✓	Complete
Reservation	✓	✓	✓	✓	✓	✓	Complete
Membership	✓	✓	✓	—	✓	✓	Complete
Wallet	✓	—	—	—	✓	✓	Partial
Seat Map	✓	✓	✓	—	✓	✓	Complete

Coverage Status

Support:

- • Complete
- • Partial
- • Planned
- • Restricted
- • Not Applicable
- • Gap

Function Drilldown

Selecting:

Ticket Product

should show:

UI Functions

- Create
- Edit
- Publish
- Suspend
- Archive

API Coverage

- GET
- POST
- PATCH
- Publish Action
- Suspend Action

Events

- ticket.product.created
- ticket.product.updated
- ticket.product.published

Actions

[View Coverage](#) | [View API](#) | [View Gap](#) | [Request API Coverage](#) | [Export Matrix](#)

Screen 2 — Ticketing & Reservation API Coverage

Purpose

Provide detailed API coverage governance for TICVAI's core ticketing and reservation capabilities.

Business Areas

Include:

Ticketing

- Ticket Products
- Ticket Types

- Ticket Variants
- Ticket Validity
- Ticket Entitlements
- Ticket Media
- Ticket Issuance
- Ticket Status
- Ticket Lifecycle

Orders

- Cart
- Orders
- Transactions
- Ticket Fulfillment

Reservations

- Availability
- Reservation Creation
- Modification
- Cancellation
- Expiration
- Holds
- Capacity

Events

- Event Products
- Performances
- Schedules
- Timeslots

Coverage Table

Function	API	Read	Write	Execute	Event	Status
----------	-----	------	-------	---------	-------	--------

Example Drilldown

Reservation — Create

UI Available: ✓

API Available: ✓

Endpoint:

- POST /reservations
- Event:
 - reservation.created
- Version:
 - v2
- Status:

Production

Gap Detection

Example:

Coverage Gap

Bulk reservation modification is available to administrators but currently has no public API.

Actions

[View Endpoint](#) | [View Event](#) | [View Documentation](#) | [Create Coverage Requirement](#)

Rule

This screen does not configure the ticket product itself. It governs **whether ticketing functionality is available programmatically**.

Screen 3 — Membership, Loyalty & Wallet API Coverage

Purpose

Govern API and business-event coverage for TICVAI's customer-value ecosystem.

Membership Coverage

Include:

- Create Membership
- Retrieve Membership
- Activate
- Renew
- Suspend
- Upgrade
- Cancel
- Benefits
- Membership Usage

Loyalty Coverage

Include:

- Loyalty Account
- Points Balance
- Earn Points
- Redeem Points
- Tier
- Benefits
- Loyalty Transactions

Wallet Coverage

Include:

- Wallet Creation
- Wallet Balance
- Credit

- • Debit
- • Transaction History
- • Refund
- • Stored Value
- • Entitlement Usage

Example Matrix

Function	Read	Create	Update	Execute	Events	Coverage
----------	------	--------	--------	---------	--------	----------

Sensitive Operation Identification

Financially sensitive actions such as:

Wallet Credit

or:

Manual Balance Adjustment

should clearly display:

Restricted / Elevated Scope Required

Detailed authorization remains Board 3.

Actions

[View API](#) | [View Event](#) | [View Scope](#) | [Review Gap](#)

Screen 4 — Access & Resource API Coverage

Purpose

Govern API coverage for operational systems including access control, venue resources, seating, and reservations.

Access Control

Include:

- • Ticket Validation
- • Admission
- • Entry
- • Exit
- • Re-entry
- • Anti-Passback
- • Gate Status
- • Credential Validation
- • Access History

Resource Management

Include:

- • Resource Availability
- • Resource Reservation
- • Resource Allocation

- • Capacity
- • Maintenance Status

Seating

Include:

- • Seat Maps
- • Sections
- • Rows
- • Seats
- • Seat Availability
- • Seat Hold
- • Seat Reservation
- • Seat Release
- • Accessibility Seats

Example

Seat Hold

API:

- POST /seats/holds
- Event:
 - • seat.hold.created
 - • Real-Time:

Required

Operational Classification

Show:

- • Real-Time Critical
- • Transactional
- • Standard
- • Background

This is useful because an access validation API has very different operational requirements from a reporting API.

Actions

[View API](#) | [View SLA Reference](#) | [View Event](#) | [Review Coverage](#)

Screen 5 — CRM & Customer API Coverage

Purpose

Govern APIs exposing customer, CRM, segmentation, and engagement functionality.

Customer Capabilities

Include:

- • Customer Profile
- • Customer Search

- • Contact Details
- • Customer Preferences
- • Customer Relationships
- • Customer History

CRM Capabilities

Include:

- • Customer Segments
- • Customer Tags
- • Interaction History
- • Campaign Association
- • Customer Notes where allowed
- • Customer Activity

Privacy Classification

Each capability should display a data classification:

Public

Internal

Personal Data

Sensitive

Restricted

Example

Customer Search API

Coverage: ✓

Data Classification:

Personal Data

Required Scope:

- customer.read
- Masking:

Policy Controlled

Data Governance Indicator

Show whether API exposure is:

- • Full
- • Masked
- • Restricted
- • Not Exposed

Actions

[View API](#) | [View Data Policy](#) | [View Scope](#) | [Review Coverage Gap](#)

Important Rule

API availability must never override TICVAI's privacy, consent, tenant-isolation, or data-protection rules.

Screen 6 — F&B, Retail & Commerce API Coverage

Purpose

Govern API coverage for TICVAI commerce operations beyond ticketing.

F&B

Include:

- Menu
- PLU
- Availability
- Orders
- Modifiers
- Fulfillment
- Kitchen Status where applicable

Retail

Include:

- Products
- Inventory
- Price
- Stock
- Orders
- Returns

Commerce

Include:

- Cart
- Pricing
- Tax
- Fees
- Payments
- Refunds
- Discounts
- Receipts

Inventory / Procurement Where Exposed

Include:

- Stock Level
- Stock Movement
- Purchase Order Status
- Supplier Information where permitted

Coverage Matrix

Capability	API	Event	Real-Time	Access	Coverage
------------	-----	-------	-----------	--------	----------

Example**Inventory Stock Update**

API: ✓

Event: ✓

Real-Time: ✓

Event:

inventory.stock.changed

Actions[View Endpoint](#) | [View Event](#) | [Review Gap](#) | [View Documentation](#)**Screen 7 — Pricing, Reporting & AI API Coverage****Purpose**

Govern programmatic access to TICVAI pricing, reporting, analytics, and AI services.

Pricing

Include:

- • Price Lookup
- • Pricing Rules
- • Pricing Calendar
- • Dynamic Pricing
- • Price Simulation
- • Price Recommendation where exposed

Reporting

Include:

- • Report Execution
- • Report Data
- • Dashboard Data
- • KPI Retrieval
- • Export
- • Scheduled Report Status

AI Services

Where approved, include APIs for:

- • AI Recommendations
- • Demand Forecasts
- • Revenue Forecasts
- • Recommendation Engine
- • AI Search / Query
- • AI Configuration Assistance
- • Risk/Fraud Scores where appropriate

AI API Governance

Each AI service should show:

Model/Service**Input Type****Output Type****Real-Time / Batch****Explainability Available****Confidence Available****API Status****Example****Customer Recommendation API**

POST /recommendations/customer/{customerId}

Returns:

- • Recommended Product
- • Recommendation Score
- • Confidence
- • Explanation where supported

Actions

[View API](#) | [Test in Sandbox](#) | [View Documentation](#) | [Review Coverage](#)

Boundary

This screen exposes the AI service through APIs. It does not configure the underlying AI engine or recommendation strategy.

Screen 8 — Partner Developer Portal**Purpose**

Provide approved TICVAI technology partners and system integrators with a dedicated workspace for managing their integration relationship with TICVAI.

This is more than the general documentation portal from Board 1.

Board 1 answers:

How do I use TICVAI APIs?

Screen 8 answers:

What integrations does my organization have with TICVAI, and where are they in the development/certification lifecycle?

Partner Header

Show:

- **Partner:** Integration Company X
- **Partner Type:** System Integrator
- **Status:** Approved
- **Partner Since:** 2026

Integration Portfolio

Integration	Type	Sandbox	Certification	Production	Status
-------------	------	---------	---------------	------------	--------

Partner Resources

Provide access to:

- Developer Documentation
- API Catalog
- Sandbox
- Test Credentials
- Certification
- Integration Guides
- Release Notes
- Support

Integration Registration

Partner can submit:

- Integration Name
- Integration Type
- Description
- TICVAI Modules
- Target Customers
- APIs Required
- Events Required
- Environment Requirements

Integration Lifecycle

Registered

→ Development

→ Sandbox Testing

→ Certification

→ Approved

→ Production

→ Marketplace Published

Actions

[Register Integration](#) | [Open Sandbox](#) | [Submit for Certification](#) | [View Findings](#) | [Manage Integration](#)

Screen 9 — Integration Certification Program

Purpose

Provide a governed certification process to validate that third-party integrations meet TICVAI's technical, security, functional, and operational standards before being approved for production or marketplace publication.

Certification Dashboard

Header KPIs:

- Certification Requests
- In Review
- Testing
- Passed
- Failed
- Conditional Approval
- Expiring Certifications

Certification Requirements

Categories:

API Compliance

- Correct endpoints
- Supported versions
- Error handling
- Idempotency

Security

- Credential handling
- Authentication
- Scope usage
- Data protection

Performance

- Acceptable traffic behavior
- Rate-limit compliance
- Timeout/retry behavior

Webhook

- Signature validation
- Retry handling
- Duplicate-event handling

Business Validation

- Correct ticket creation
- Correct payment behavior
- Correct refunds
- Correct access validation where applicable

Certification Test Plan

Example:

Partner Reseller Integration

Tests:

1. • Retrieve Availability
2. • Create Reservation
3. • Complete Order
4. • Issue Ticket
5. • Cancel Reservation
6. • Process Refund
7. • Receive ticket.sold
8. • Handle duplicate webhook
9. • Handle API timeout
10. • Handle rate-limit response

Test Result

Each test:

Passed / Failed / Warning / Not Applicable

Findings

Track:

- • Finding
- • Severity
- • Owner
- • Due Date
- • Evidence
- • Resolution
- • Retest Result

Certification Decision

- Approved
- Approved with Conditions
- Rejected
- Expired
- Suspended

Certification Validity

Where required:

Valid Until: configurable date

Actions

Start Review | Run Tests | Add Finding | Request Fix | Retest | Approve | Reject

Important Integration

Testing should reuse capabilities from **Board 7 — Sandbox & Integration Testing** rather than building a duplicate testing engine.

Screen 10 — API & Integration Marketplace

Purpose

Provide TICVAI with a governed catalog where approved integrations, connectors, partner solutions, and API-based extensions can be discovered by TICVAI clients.

This should operate as a **controlled ecosystem marketplace**, not an unrestricted public app store.

Marketplace Categories

Examples:

- • Payment
- • CRM
- • ERP
- • Accounting
- • Access Control
- • Hardware
- • Marketing
- • Resellers
- • Hospitality
- • Analytics
- • Identity
- • Mobile
- • Government
- • AI
- • Other

Integration Cards

Each card should show:

Integration Name

Partner

Category

Certified ✓

Supported TICVAI Modules

Supported Regions

Current Version

Integration Detail

Show:

- • Description
- • Partner
- • Integration Type
- • TICVAI Modules
- • APIs Used
- • Business Events Used

- Requirements
- Supported Versions
- Certification Status
- Documentation
- Support Information
- Release History

Compatibility

Example:

TICVAI Compatibility

Ticketing ✓

Membership ✓

Wallet ✓

Access Control —

Retail —

Marketplace Status

Support:

- Draft
- Submitted
- Under Review
- Certified
- Published
- Suspended
- Deprecated
- Retired

Publication Workflow

Partner Submission

↓

Technical Certification

↓

TICVAI Review

↓

Approval

↓

Marketplace Publication

Actions

[View Integration](#) | [Request/Enable Integration](#) | [View Documentation](#) | [Contact Partner](#) | [View Certification](#)

Actual commercial purchasing or subscription activation should integrate with TICVAI's Subscription & Licensing capabilities rather than being duplicated here.

Critical API Coverage Principle

This is one of the most important architectural requirements for Board 8.

Soft Labs should **not manually maintain two unrelated lists**:

- UI functionality
- and:
- • API functionality.
- • TICVAI should maintain a structured relationship between:

Business Capability

↓

Owning TICVAI Module

↓

UI Action

↓

API Endpoint / Operation

↓

Business Event

↓

API Version

↓

Access Scope

For example:

Business Capability — Refund Ticket

Owning Module: Ticketing / Finance

UI Action: Refund

API:

- POST /tickets/{id}/refund
- **Scope:**
- • ticket.refund.execute
- • **Business Event:**
- • • ticket.refunded
- • • **Version:**
- • • • v2
- • • • **Coverage:**

Complete ✓

This gives TICVAI a measurable answer to:

“Can everything we can do through the administration portal also be done through an API?”

API Coverage Gap Management

When TICVAI detects:

UI Function Exists = YES

but:

API Capability = NO

the platform should identify:

Coverage Gap

Example:

Business Function: Bulk Membership Extension

UI: Available

API: Not Available

Event: Available

Coverage: Partial

The administrator can then create a development requirement rather than discovering the gap only when a customer asks for the integration.

This is especially valuable as TICVAI grows.

Partner Integration Health

Although detailed runtime health remains in Board 4, the Partner Portal should summarize integration status.

Example:

Dubai CRM Connector

Certification: ✓ Valid

Production: Active

API Version: v3

Health: Healthy

Last API Activity: 3 min ago

Current Issues: 0

Clicking **Health** should deep-link to Board 4 rather than reproducing its monitoring screens.

Certification Expiry & API Lifecycle

Certification must also connect with **Board 6**.

Example:

Partner integration certified against:

Ticketing API v2

TICVAI releases:

Ticketing API v3

Board 6 identifies migration requirement.

Board 8 can then show:

- ⚠ **Recertification Required**
- **Current Certification:** v2
- **Target:** v3

- **Deadline:** 31 December 2026

This prevents the marketplace from showing integrations as fully certified when they only support retired API versions.

Cross-Board Boundaries

Function	Owner
API documentation/discovery	Board 1
Developer identity/credentials	Board 2
API access/security/scopes	Board 3
API monitoring/integration health	Board 4
Webhooks/business events	Board 5
API versions/migrations/releases	Board 6
Sandbox/testing	Board 7
Business API coverage governance	Board 8
Partner integration workspace	Board 8
Integration certification	Board 8
Integration marketplace	Board 8
Product/business configuration	Owning TICVAI Module
Commercial licenses/subscriptions	Subscription & Licensing Module

Board 8 — Final Screen Map

Business API Coverage & Partner Ecosystem

1. Business API Coverage Matrix
2. Ticketing & Reservation API Coverage
3. Membership, Loyalty & Wallet API Coverage
4. Access & Resource API Coverage
5. CRM & Customer API Coverage
6. F&B, Retail & Commerce API Coverage
7. Pricing, Reporting & AI API Coverage
8. Partner Developer Portal
9. Integration Certification Program
10. API & Integration Marketplace

With Board 8, the **Developer & API Management** module is complete: **8 boards / 80 screens**. The eight boards now cover the full path from **developer discovery** → **identity and credentials** → **security** → **monitoring** → **events** → **lifecycle** → **sandbox/testing** → **business API coverage and partner ecosystem**.